

计算理论作业

胡临天

2026 年 5 月 2 日

目录

Chapter 0	2
Chapter 1	6
Chapter 2	42
Chapter 3	43
Chapter 4	44
Chapter 5	45
Chapter 6	46
Chapter 7	47
Chapter 8	64
Chapter 9	65
Chapter 10	66

Chapter 0

0.4

0.4 考察下列集合的形式化描述，先理解它们包括什么样的元素，写一段简短文字描述每一个集合。

- a. $\{1, 3, 5, 7, \dots\}$ 。
- b. $\{\dots, -4, -2, 0, 2, 4, \dots\}$ 。
- c. $\{n \mid \text{对 } \mathcal{N} \text{ 中的某一个 } m, n=2m\}$ 。
- d. $\{n \mid \text{对 } \mathcal{N} \text{ 中的某一个 } m, n=2m, \text{ 并且对 } \mathcal{N} \text{ 中的某个 } k, n=3k\}$ 。
- e. $\{w \mid w \text{ 是 } 0、1 \text{ 字符串, 且 } w \text{ 等于 } w \text{ 的反转}\}$ 。
- f. $\{n \mid n \text{ 是一个整数, 且 } n=n+1\}$ 。

图 1

解答.

- (a) 这是所有正奇数组成的集合，即 $1, 3, 5, 7, \dots$ 。
- (b) 这是所有偶数组成的集合，包括负偶数、0 和正偶数。
- (c) 这是所有能写成 $2m$ 的自然数组成的集合，也就是所有偶自然数。
- (d) 这是所有同时是 2 的倍数和 3 的倍数的自然数组成的集合，也就是所有 6 的倍数。
- (e) 这是所有由 0,1 构成并且满足 $w = w^R$ 的字符串组成的集合，也就是所有二进制回文串。
- (f) 这是空集，因为不存在整数 n 满足 $n = n + 1$ 。

案:

0.7

0.7 对下列每一小题，给出一个满足指定条件的关系。

- a. 自反的和对称的，但不是传递的。
- b. 自反的和传递的，但不是对称的。
- c. 对称的和传递的，但不是自反的。

图 2

案: 可以把关系看成定义在某个顶点集上的一组有向边。这样一来，自反性对应于每个顶点都有一个自环；对称性对应于若存在边 $x \rightarrow y$ ，则也必须存在边 $y \rightarrow x$ ；传递性对应于只要存在路径 $x \rightarrow y \rightarrow z$ ，就必须再有一条边 $x \rightarrow z$ 。因此，从图的视角来看，这道题实际上是在构造满足若干边性质、但不满足另一些边性质的小型有向图。这个视角对于寻找反例和构造例子都很有帮助。

0.9

0.9 试写出下述图的形式化描述

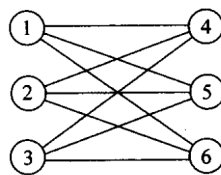


图 3

解答.

$$(\{1, 2, 3, 4, 5, 6\}, \{(1, 4), (1, 5), (1, 6), (2, 4), (2, 5), (2, 6), (3, 4), (3, 5), (3, 6)\})$$

案: 图的形式化描述的方法是 (V, E) , 有此约定能通过顶点和边的二元组来形式化描述一个图, 同样通过这个约定能从形式化的描述中画出对应的图。

0.10

0.10 请找出下述证明 $2=1$ 中的错误。

提示: 考虑到方程 $a=b$ 。两边同乘以 a , 得到 $a^2=ab$ 。两边同时减去 b^2 , 得到 $a^2-b^2=ab-b^2$ 。对每一边做因式分解, 得 $(a+b)(a-b)=b(a-b)$ 。用 $(a-b)$ 除每一边, 得 $a+b=b$ 。最后, 令 a 和 b 等于 1, 得证 $2=1$ 。

图 4

解答. 证明 “ $2=1$ ” 的错误所在:

原始论证复现

设 $a=b$, 则:

1. $a=b$ (前提)
2. $a^2=ab$ (两边乘以 a)
3. $a^2-b^2=ab-b^2$ (两边减去 b^2)
4. $(a+b)(a-b)=b(a-b)$ (两边因式分解)
5. $a+b=b$ (两边除以 $(a-b)$) $\leftarrow$ 错误在此
6. 令 $a=b=1$, 得 $2=1$

错误出现在第 4 步到第 5 步的推导中。

由前提 $a=b$, 可得:

$$a-b=0$$

第 5 步的操作是将等式两边同除以 $(a-b)$, 即除以 0。

除以零是未定义的操作。在实数域 (或任何域) 的公理中, 乘法逆元的存在性要求:

$$\forall x \in \mathbb{R}, x \neq 0 \implies \exists x^{-1} \text{ 使得 } x \cdot x^{-1} = 1$$

当 $x = 0$ 时, 不存在这样的逆元。因此, “两边除以 $(a - b)$ ” 这一步骤没有合法性, 后续一切推导均无效。

案 该“证明”通过在 $a = b$ 的前提下对 $(a - b) = 0$ 做除法, 违反了实数域中除法要求除数非零的基本规则, 从而导出了矛盾。错误不在代数变换本身, 而在于施加了一个不合法的运算。■

0.11

0.11 请找出下述证明中的错误, 它证明所有的马颜色相同。

断言: 在任意一个 h 匹马的集合中, 所有的马颜色相同。

证明: 对 h 作归纳证明。

1) 归纳基础: 对于 $h=1$, 在任何只有一匹马的集合中, 显然所有的马颜色相同。

2) 归纳步骤: 对于 $k \geq 1$ 。假设命题对于 $h=k$ 为真, 要证命题对于 $h=k+1$ 也为真。

令 H 为任意一个有 $k+1$ 匹马的集合, 现在要证明这个集合中的所有马颜色相同。

图 5

从这个集合中牵走一匹马, 得到集合 H_1 , H_1 中恰好有 k 匹马。根据归纳假设, H_1 中所有的马颜色相同。现在把牵走的马重新牵回来, 再牵走另外一匹马得到集合 H_2 。根据同样的道理, H_2 中所有的马颜色也应相同。因此, H 中所有的马颜色一定相同。

图 6

解答. “所有马颜色相同”的错误证明——严格分析

原始论证复现

断言: 在任意一个 h 匹马的集合中, 所有的马颜色相同。

“证明”: 对 h 作归纳。

1. 归纳基础: $h = 1$ 时, 只有一匹马, 显然颜色相同。✓

2. 归纳步骤: 假设对 $h = k$ 命题为真, 证明对 $h = k + 1$ 也为真。

令 $H = \{h_1, h_2, \dots, h_{k+1}\}$ 为任意 $k + 1$ 匹马的集合。构造:

$$H_1 = \{h_1, h_2, \dots, h_k\}, \quad H_2 = \{h_2, h_3, \dots, h_{k+1}\}$$

由归纳假设, H_1 中所有马颜色相同, H_2 中所有马颜色相同。

而 $H_1 \cap H_2 = \{h_2, \dots, h_k\}$ 非空, 故 H_1 和 H_2 的马颜色一致, 从而 H 中所有马颜色相同。

错误所在

错误出现在归纳步骤从 $k = 1$ 到 $k = 2$ 的过渡中。

当 $k = 1$ 时, 需要证明任意 $k + 1 = 2$ 匹马颜色相同。令 $H = \{h_1, h_2\}$, 按上述方法构造:

$$H_1 = \{h_1\}, \quad H_2 = \{h_2\}$$

此时：

$$H_1 \cap H_2 = \emptyset$$

交集为**空集**。因此无法通过公共元素将 H_1 和 H_2 的颜色“桥接”起来。归纳步骤所依赖的关键推理

“ $H_1 \cap H_2$ 非空，故两个子集的马颜色一致”

——在 $k = 1$ 时**不成立**。

严格说明

归纳步骤的论证隐含地使用了如下事实：

$$|H_1 \cap H_2| = |H| - 2 = k + 1 - 2 = k - 1$$

这要求 $k - 1 \geq 1$ ，即 $k \geq 2$ ，交集才非空。

因此，归纳步骤仅在 $k \geq 2$ 时有效。而从基础 $h = 1$ 到 $h = 2$ 的这一步恰好是归纳链断裂的地方——整个归纳证明在最关键的第一步就失败了。

案：该证明的错误是一个**归纳步骤的适用范围错误**：归纳步骤要求两个 k 元子集有非空交集（即 $k \geq 2$ ），但归纳基础仅建立了 $k = 1$ 的情形。从 $k = 1$ 到 $k = 2$ 的推导无法完成，归纳链在起点即断裂。■

0.13

***0.13 Ramsey 定理。**设 G 是一个图， G 中的**团**为任意两个顶点都有边相连的子图。**反团**又叫做**独立集**，它为任意两个顶点都没有边相连的子图。试证明：所有 n 个顶点的图都包含一个顶点数不少于 $\frac{1}{2} \log_2 n$ 的团或反团。

图 7

解答：

0.13 将空间划分成两类顶点集， A 和 B 。然后扫描整个图，如果顶点 x 度数大于空间中剩余顶点数目的一半，则把 x 加到 A 中，否则将其加到 B 中。如果 x 加到 A 中，则不再考虑所有不与其相连的顶点；如果 x 加到 B 中，则不再考虑所有与其相连的顶点。重复这个步骤，直到处理完所有顶点。由于每一步最多丢弃一半的顶点，因此至少需要经过 $\log_2 n$ 步过程才会终止。由于每步只在 A 或 B 中添加一个顶点，所以在过程结束后 A 或 B 中至少含有 $\frac{1}{n} \log_2 n$ 个顶点。其中， A 中包含的顶点属于团， B 中包含的顶点属于反团。

图 8

案：这题是通过构造性证明的方式来证明：所有 n 个顶点的图都包含一个顶点数不少于 $\frac{1}{2} \log_2 n$ 的**团或反团**。通过观察构造出来的算法是如何处理 n （节点的个数）证明的这个定理。算法设计用于解决问题，从图灵机的视角来看就是在识别“语言”，反过来分析算法的步骤能得出更多的这个“语言”的性质。如果进一步利用这个性质，或能用于优化这个算法，或能用于优化其他算法。分析做是从另一个视角看一个问题（语言），优化做的是对算法的重新理解。

Chapter 1

1.4

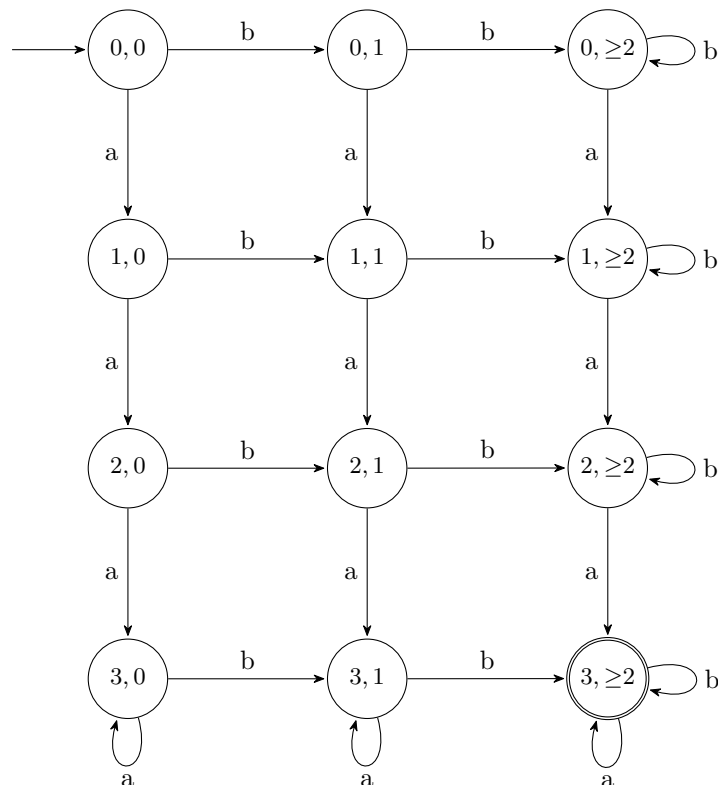
1.4 下面每个语言都是两个简单语言的交。对每一小题先构造简单语言的 DFA，然后按照 1.1.5 节的脚注所讨论的构造方法结合它们以画出给定语言的 DFA 状态图。在所有问题中 $\Sigma = \{a, b\}$ 。

- a. $\{w \mid w \text{ 含有至少 3 个 } a \text{ 和至少 2 个 } b\}$
- ^A b. $\{w \mid w \text{ 含有正好 2 个 } a \text{ 和至少 2 个 } b\}$
- c. $\{w \mid w \text{ 含有偶数个 } a \text{ 和 1 个或 2 个 } b\}$
- ^A d. $\{w \mid w \text{ 含有偶数个 } a \text{ 并且每个 } a \text{ 后都跟有至少一个 } b\}$
- e. $\{w \mid w \text{ 从 } a \text{ 开始并且最多有 1 个 } b\}$
- f. $\{w \mid w \text{ 含有奇数个 } a \text{ 并且以 } b \text{ 结束}\}$
- g. $\{w \mid w \text{ 的长度为偶数并且有奇数个 } a\}$

图 9

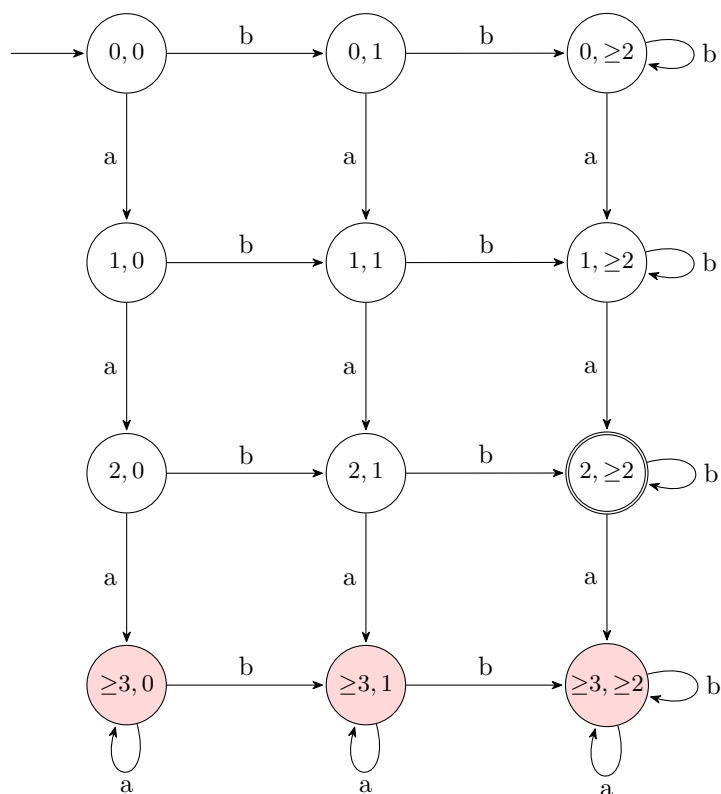
(a) $\{w \mid w \text{ 含至少 3 个 } a \text{ 和至少 2 个 } b\}$

状态 $(i, j): i = a \text{ 的计数 } (3 \text{ 饱和}), j = b \text{ 的计数 } (2 \text{ 饱和})$ 。接受态 $(3, \geq 2)$ 。



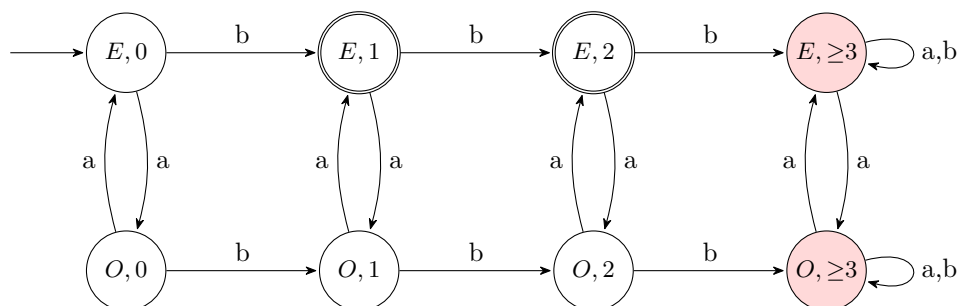
(b) $\{w \mid w \text{ 含正好 2 个 } a \text{ 和至少 2 个 } b\}$

第 4 行 $(\geq 3, \cdot)$ 为死状态 (红色), 接受态仅 $(2, \geq 2)$ 。



(c) $\{w \mid w \text{ 含偶数个 } a \text{ 和 } 1 \text{ 个或 } 2 \text{ 个 } b\}$

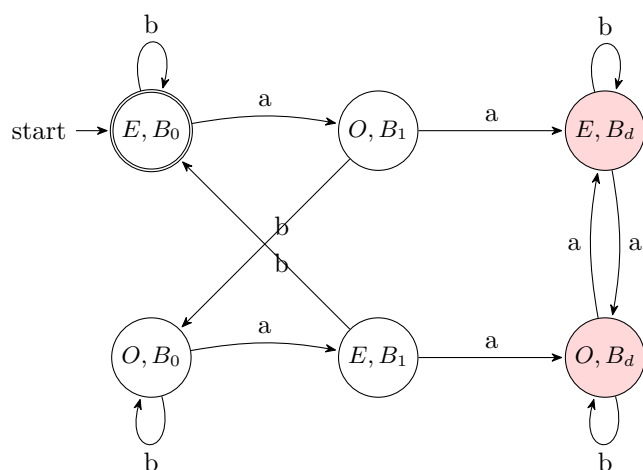
行: a 奇偶 (E/O); 列: b 计数 0, 1, 2, ≥ 3 (死)。接受态 (E, 1) 和 (E, 2)。



注: 死列箭头 a 实际指向另一行的死态, 本图已用两条双向箭头覆盖。

(d) $\{w \mid w \text{ 含偶数个 } a \text{ 且每个 } a \text{ 后跟至少一个 } b\}$

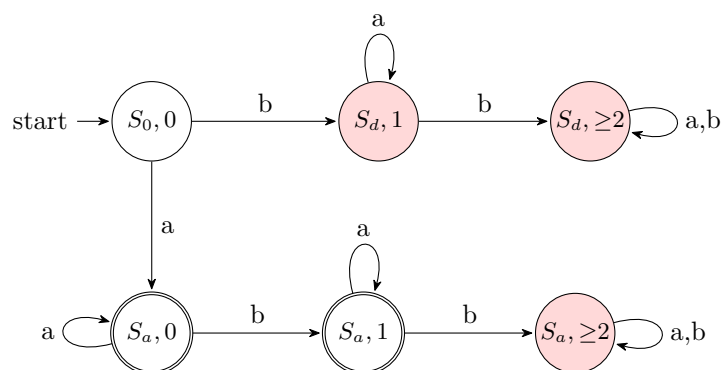
行: a 奇偶 E/O; 列: B_0 (可接受)、 B_1 (刚读 a 等 b)、 B_d 死。唯一接受 (E, B_0)。



案:

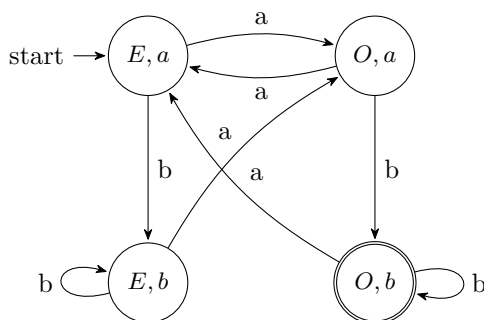
(e) $\{w \mid w \text{ 从 } a \text{ 开始且最多 } 1 \text{ 个 } b\}$

只绘从起始态可达的 6 个状态。接受态 $(S_a, 0)$ 和 $(S_a, 1)$ 。



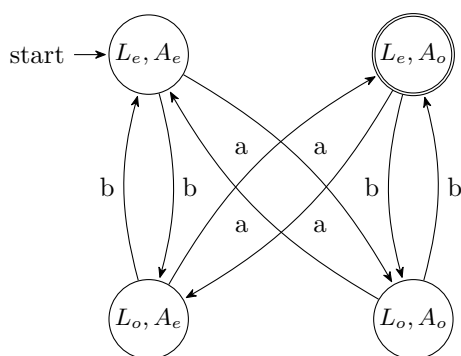
(f) $\{w \mid w \text{ 含奇数个 } a \text{ 且以 } b \text{ 结束}\}$

状态:(a 奇偶, 末字符)。起始态 (E, a) (空串归此类)。接受态 (O, b) 。



(g) $\{w \mid |w| \text{ 为偶数且 } w \text{ 有奇数个 } a\}$

状态:(长度奇偶, a 奇偶)。任何字符都翻转长度; a 额外翻转 a 计数。接受态 (L_e, A_o) 。



观察: 由 $|w| = \#_a + \#_b, |w|$ 偶且 $\#_a$ 奇 $\iff \#_b$ 亦为奇数, 所以此语言等价于 $\{w \mid \#_a, \#_b \text{ 均为奇数}\}$ 。

案: “和”、“并且”等表示的是

1.5

1.5 下述每个语言都是一个简单语言的补。对每一小题先构造简单语言的 DFA, 然后用其画出给定语言的 DFA 状态图。在所有问题中 $\Sigma = \{a, b\}$ 。

^A a. $\{w \mid w \text{ 中不含子串 } ab\}$

图 10

^A b. $\{w \mid w \text{ 中不含子串 } baba\}$

c. $\{w \mid w \text{ 中既不含子串 } ab \text{ 也不包含子串 } ba\}$

d. $\{w \mid w \text{ 是不在 } a^* b^* \text{ 中的任意串}\}$

e. $\{w \mid w \text{ 是不在 } (ab^+)^* \text{ 中的任意串}\}$

f. $\{w \mid w \text{ 是不在 } a^* \cup b^* \text{ 中的任意串}\}$

g. $\{w \mid w \text{ 是恰好不含 2 个 } a \text{ 的任意串}\}$

h. $\{w \mid w \text{ 是除 } a \text{ 和 } b \text{ 外的任意串}\}$

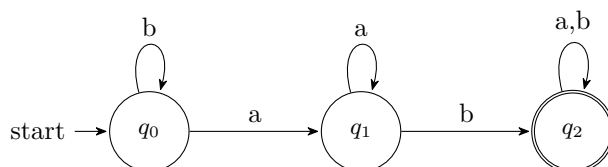
图 11

解答: 方法总结: 对每小题先给简单语言 L_1 构造完备 DFA M (所有状态对所有字符都必须有转移, 必要时添加死状态), 然后将接受态集 F 换成 $Q \setminus F$, 得到识别 $\overline{L_1}$ 的 DFA。转移保持不变。

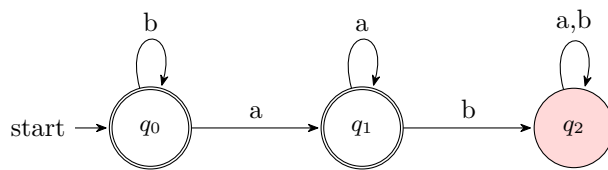
(a) $\{w \mid w \text{ 中不含子串 } ab\}$

简单语言 $L_1 = \{w \mid w \text{ 含子串 } ab\}$ 。

L_1 的 DFA:



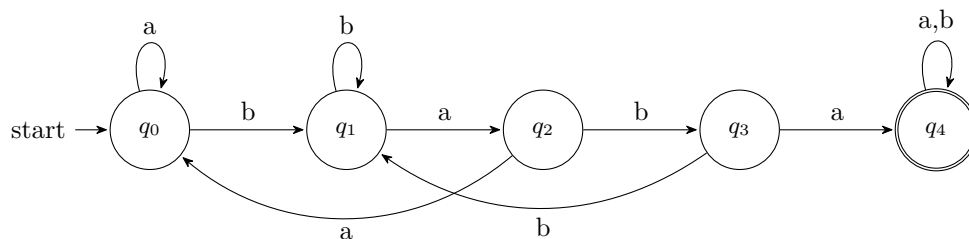
$\overline{L_1}$ 的 DFA(接受态互换):



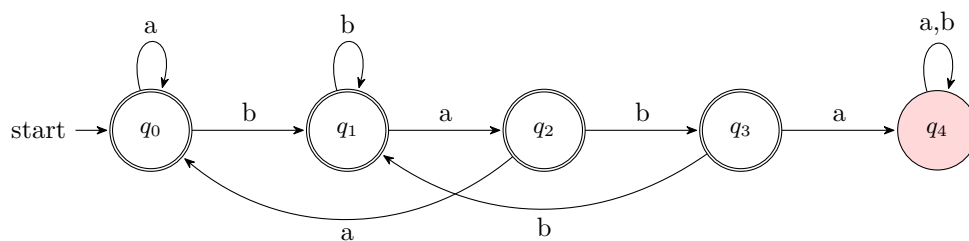
(b) $\{w \mid w \text{ 中不含子串 } baba\}$

简单语言 $L_1 = \{w \mid w \text{ 含子串 } baba\}$ 。失配回退是关键: q_3 读 b 退回 q_1 (新 b 可为新匹配开头); q_2 读 a 退回 q_0 。

L_1 的 DFA:



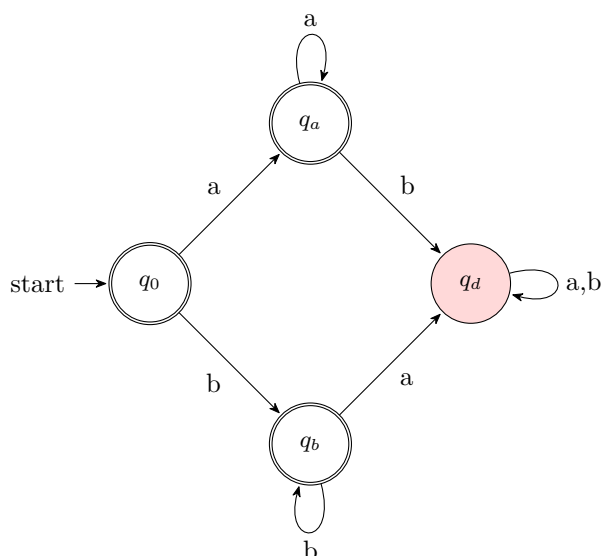
$\overline{L_1}$ 的 DFA:



(c) $\{w \mid w \text{ 中既不含 } ab \text{ 也不含 } ba\}$

等价于 $a^* \cup b^*$ 。直接构造: 4 个状态, q_d 是混合后的死状态。

直接 DFA(此题直接给出比用补集更清晰):

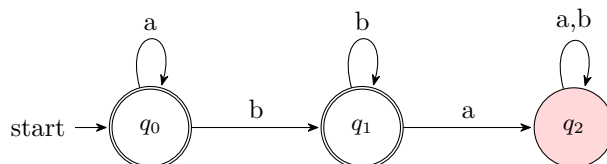


若严格按补集方法: 设 $L_1 = \{w \mid w \text{ 含 } ab \text{ 或含 } ba\}$, 则只需把上图的接受态翻转: q_0, q_a, q_b 变非接受, q_d 变接受即得 L_1 的 DFA。

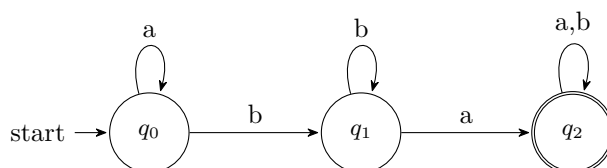
(d) $\{w \mid w \notin a^*b^*\}$

简单语言 $L_1 = a^*b^*$ 。三个状态: q_0 读 a 阶段, q_1 读 b 阶段, q_2 b 后又读 a(死)。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:

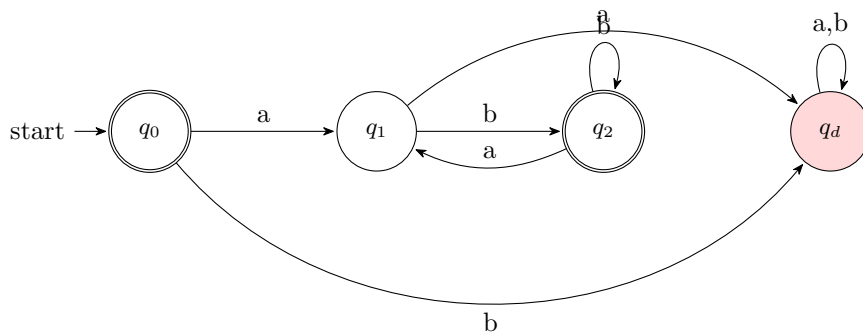


(e) $\{w \mid w \notin (ab^+)^*\}$

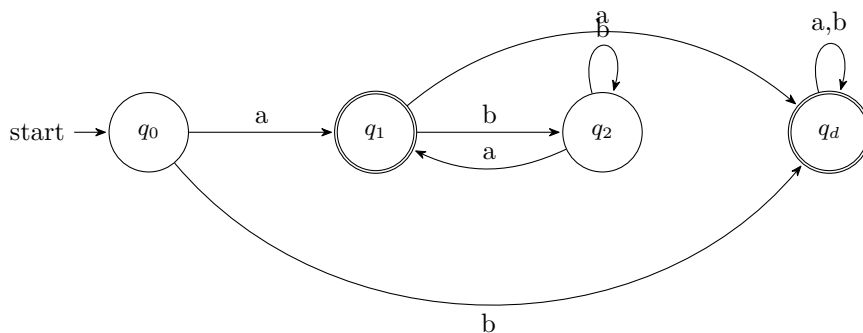
简单语言 $L_1 = (ab^+)^*$ = 零或多个"a 后跟至少一个 b" 块的拼接。

状态: q_0 起始/块完成; q_1 读 a 等 b; q_2 已读 ab(可接受, 可继续读 b 或开新块); q_d 死。

L_1 的 DFA:



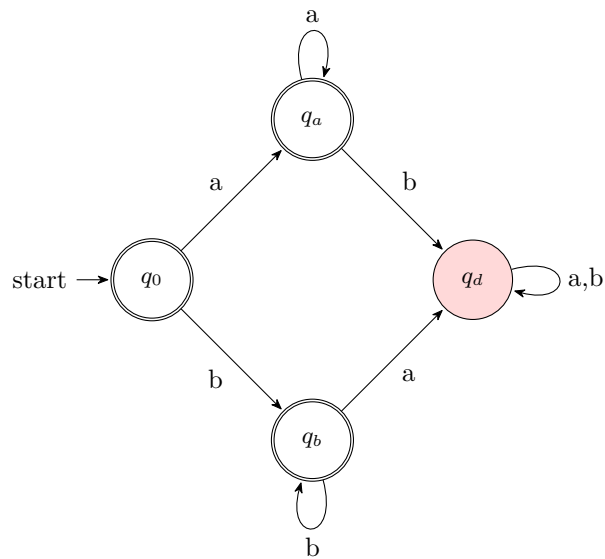
$\overline{L_1}$ 的 DFA:



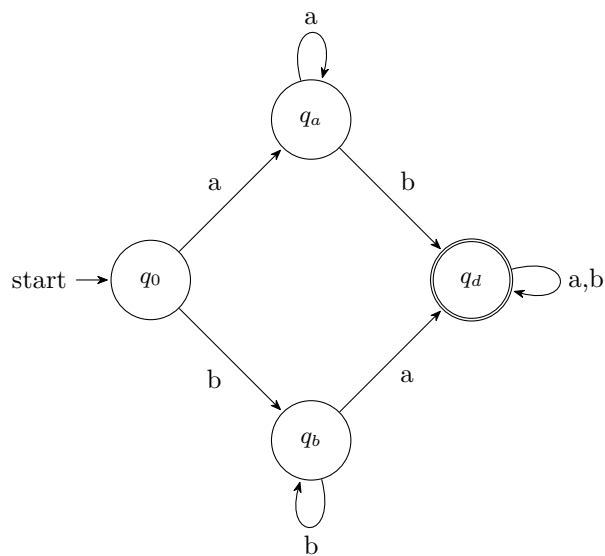
(f) $\{w \mid w \notin a^* \cup b^*\}$

简单语言 $L_1 = a^* \cup b^*$ (全 a 串或全 b 串, 含空串)。结构与 (c) 题相同。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:

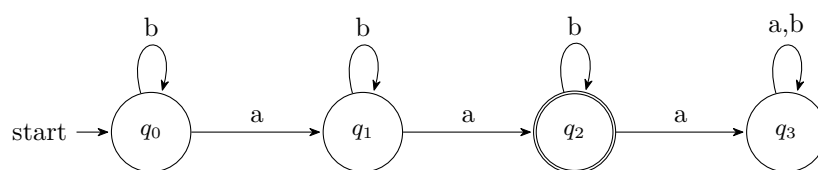


目标语言: a 和 b 都至少出现一次 (顺序不限) 的串。

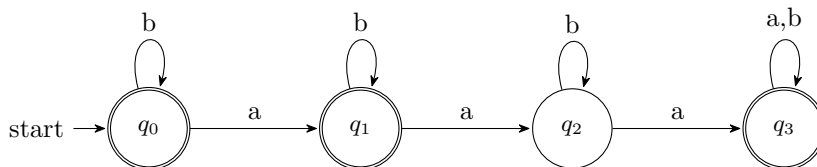
(g) $\{w \mid w \text{ 恰好不含 2 个 a}\}$

即 a 的个数 $\neq 2$ 。简单语言 $L_1 = \{w \mid w \text{ 恰好含 2 个 a}\}$ 。

L_1 的 DFA:



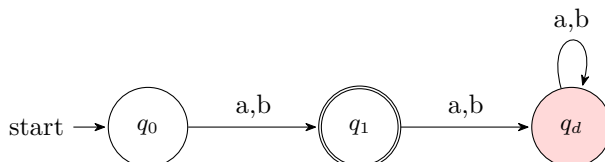
$\overline{L_1}$ 的 DFA:



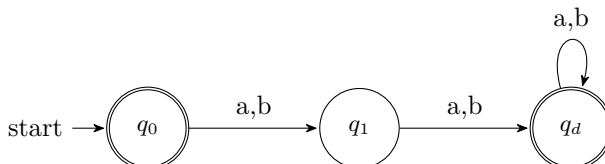
(h) $\{w \mid w \text{ 是除 } a \text{ 和 } b \text{ 外的任意串}\}$

即 $\Sigma^* \setminus \{a, b\}$ ——除了长度为 1 的两个串“a”和“b”以外的所有串 (包括空串!). 简单语言 $L_1 = \{a, b\}$ 。

L_1 的 DFA:



$\overline{L_1}$ 的 DFA:



注: q_0 接受空串 ε ; q_d 接受所有长度 ≥ 2 的串。

1.6

1.6 画出识别下述语言的 DFA 状态图。在所有问题中字母表均为 $\{0, 1\}$

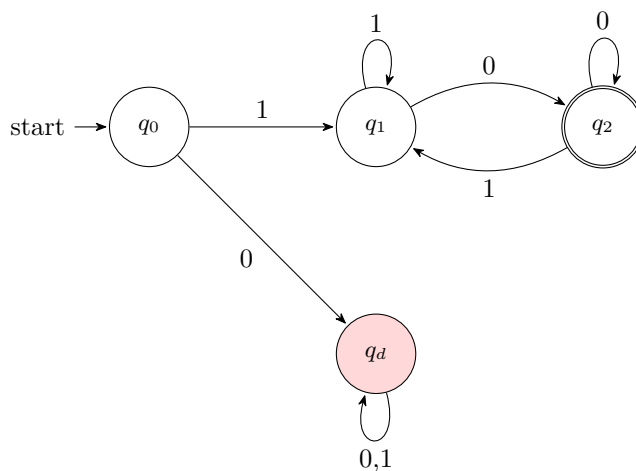
- a. $\{w \mid w \text{ 从 } 1 \text{ 开始且以 } 0 \text{ 结束}\}$
- b. $\{w \mid w \text{ 含有至少 } 3 \text{ 个 } 1\}$
- c. $\{w \mid w \text{ 含有子串 } 0101, \text{ 即对某个 } x \text{ 和 } y, w = x0101y\}$
- d. $\{w \mid w \text{ 的长度不小于 } 3, \text{ 并且第 } 3 \text{ 个符号为 } 0\}$
- e. $\{w \mid w \text{ 从 } 0 \text{ 开始且长度为奇数, 或者从 } 1 \text{ 开始且长度为偶数}\}$
- f. $\{w \mid w \text{ 不含子串 } 110\}$
- g. $\{w \mid w \text{ 的长度不超过 } 5\}$
- h. $\{w \mid w \text{ 是除 } 11 \text{ 和 } 111 \text{ 外的任意串}\}$
- i. $\{w \mid w \text{ 的奇数位置均为 } 1\}$
- j. $\{w \mid w \text{ 含有至少 } 2 \text{ 个 } 0, \text{ 并且至多含 } 1 \text{ 个 } 1\}$
- k. $\{\varepsilon, 0\}$
- l. $\{w \mid w \text{ 含有偶数个 } 0 \text{ 或恰好 } 2 \text{ 个 } 1\}$
- m. 空集
- n. 除空串外的所有字符串

图 12

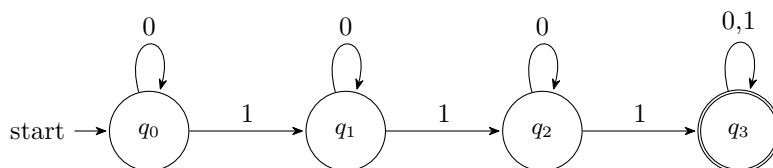
约定: 字母表 $\Sigma = \{0, 1\}$ 。所有 DFA 都是完备的, 死状态用红色底色标出, 接受态用双圆。

(a) $\{w \mid w \text{ 从 } 1 \text{ 开始且以 } 0 \text{ 结束}\}$

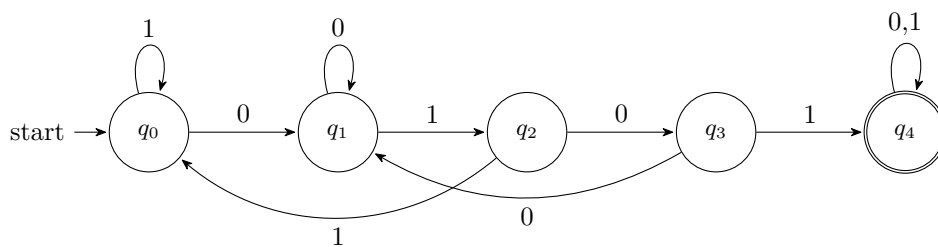
状态: q_0 起始 (未读字符); q_d 首字符为 0 的死状态; q_1 首字符 1, 当前末字符 1 (非接受); q_2 首字符 1, 当前末字符 0 (接受)。

(b) $\{w \mid w \text{ 含有至少 } 3 \text{ 个 } 1\}$

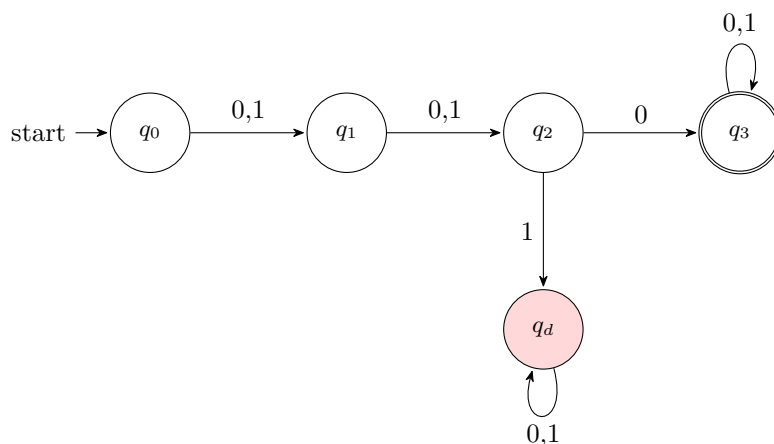
按 1 的计数划分, 3 饱和。

(c) $\{w \mid w \text{ 含子串 } 0101\}$

跟踪匹配进度, 失配时回到最长的已匹配前缀。关键回退: q_1 (已匹配 0) 读 0 保持 q_1 ; q_3 (已匹配 010) 读 0 退回 q_1 (新 0 可能是新匹配开头)。

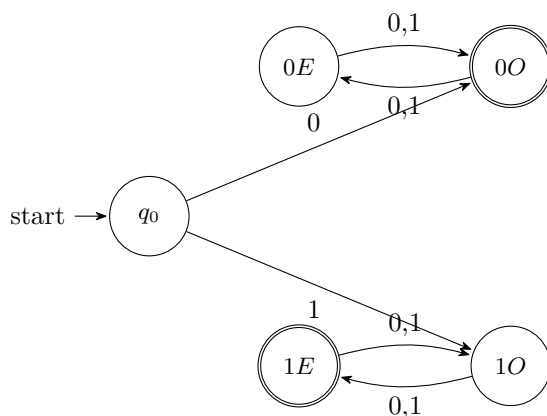
(d) $\{w \mid |w| \geq 3 \text{ 且第 } 3 \text{ 个符号为 } 0\}$

前三步按位置推进, 第 3 个字符决定去向。



(e) $\{w \mid w \text{ 从 } 0 \text{ 开始且长度奇数, 或从 } 1 \text{ 开始且长度偶数} \}$

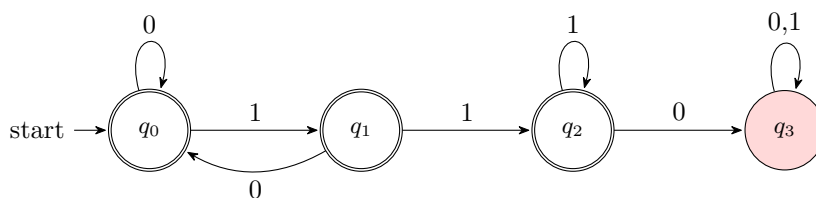
空串不满足任一条件 (空串无首字符)。5 个状态: q_0 起始; $0E, 0O$ (首 0, 当前长度偶/奇); $1E, 1O$ (首 1, 当前长度偶/奇)。接受: $0O$ (首 0 + 奇长)、 $1E$ (首 1 + 偶长, 但长度至少为 2)。



注: 从 q_0 读 0 后长度已为 1 (奇), 直接进入接受态 $0O$; 读 1 后长度为 1 (奇, 不接受), 进入 $1O$, 再读一字符才到 $1E$ 接受。

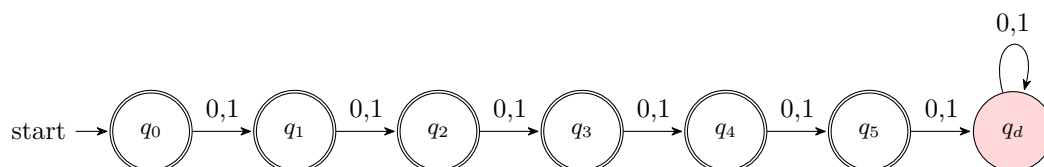
(f) $\{w \mid w \text{ 不含子串 } 110\}$

先画含 110 的 DFA, 再翻转接受态。 q_0 (未匹配)、 q_1 (匹配 1)、 q_2 (匹配 11)、 q_3 (已含 110)。



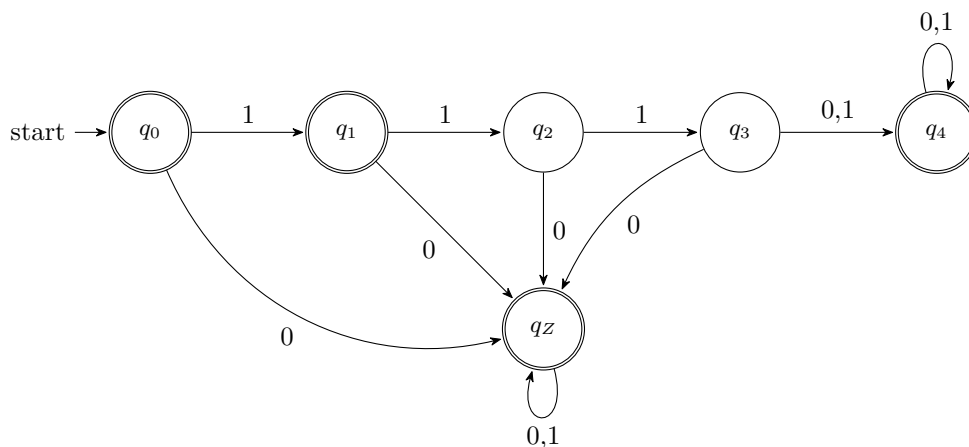
(g) $\{w \mid |w| \leq 5\}$

6 个接受态对应长度 0, 1, ..., 5, 第 7 个状态为死态 (长度 ≥ 6)。



(h) $\{w \mid w \neq 11 \text{ 且 } w \neq 111\}$

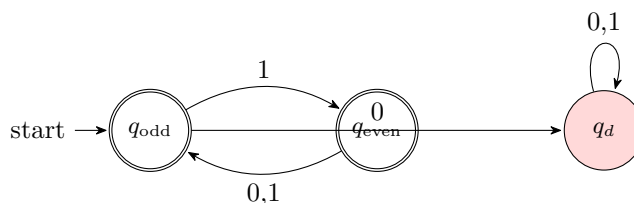
即 $\Sigma^* \setminus \{11, 111\}$ 。先识别 $\{11, 111\}$ 再取补。只需追踪“连续 1 的前缀长度 0/1/2/3”以及“是否已经出现过 0”。



状态含义: $q_0 = \varepsilon$ (接受); $q_1 = 1$ (接受, 不等于 11 或 111); $q_2 = 11$ (不接受); $q_3 = 111$ (不接受); $q_4 =$ 长度 ≥ 4 的纯 1 串或 111 之后还有字符 (接受); $q_z =$ 任何阶段出现过 0, 说明不是纯 1 串, 肯定不等于 11 或 111(接受)。

(i) $\{w \mid w \text{ 的奇数位置均为 } 1\}$

位置从 1 开始数。奇数位必须是 1, 偶数位随意。空串空位置集满足, 接受。

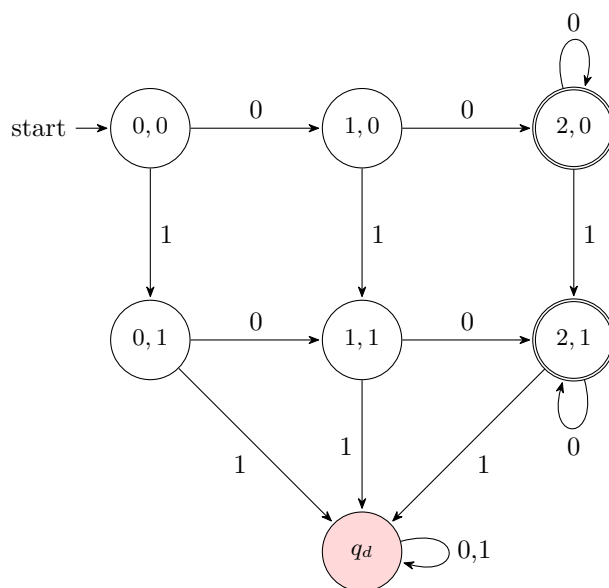


q_{odd} : 下一读入位置是奇数, 必须为 1; q_{even} : 下一位置是偶数, 0/1 都可; 一旦奇数位读到 0 就进入死态。

(j) $\{w \mid w \text{ 含至少 } 2 \text{ 个 } 0 \text{ 且至多 } 1 \text{ 个 } 1\}$

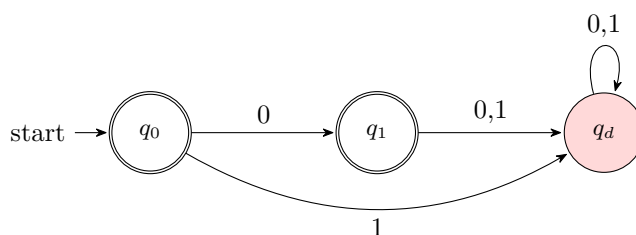
笛卡尔积: 0 计数 $\{0, 1, \geq 2\} \times 1$ 计数 $\{0, 1, \geq 2(\text{死})\}$ 。去掉 1 计数 ≥ 2 那列后只剩 2 列, 加一个共用死态共 $3 \times 2 + 1 = 7$ 个状态 (下面把同为死态的合并)。

状态记作 (i, j) , $i = 0$ 的计数 (0/1/2+), $j = 1$ 的计数 (0/1)。接受 $(2, 0)$ 和 $(2, 1)$ 。



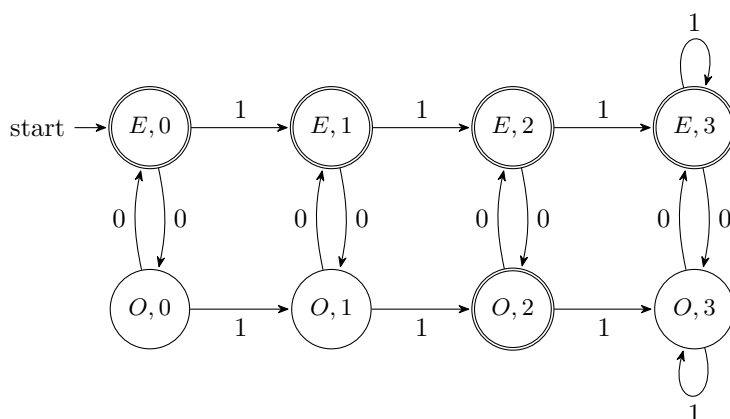
(k) $\{\varepsilon, 0\}$

只接受空串和单字符 0。



(l) $\{w \mid w \text{ 含偶数个 } 0 \text{ 或恰好 } 2 \text{ 个 } 1\}$

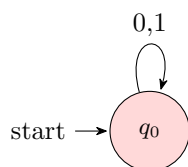
并运算用笛卡尔积: $\{w \mid \#_0 \text{ 偶}\}$ 的 2 状态 DFA $\times \{w \mid \#_1 = 2\}$ 的 4 状态 DFA = 8 状态。接受态:** 至少一个分量是接受态 ** (并运算)。状态记为 (p, k) , $p \in \{E, O\}$ (0 的奇偶), $k \in \{0, 1, 2, 3\}$ (1 的计数, 3 代表 ≥ 3)。



接受规则 (并): E 行全部接受 (因为 0 偶); O 行中只有 $(O, 2)$ 接受 (1 恰好 2 个)。

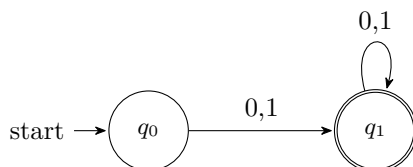
(m) 空集 $\emptyset$

不接受任何串。只需一个非接受态带所有字符自环。



(n) 除空串外的所有串 Σ^+

起始态不接受 (用于排除空串), 读任一字符就进入永久接受态。



1.7

1.7 给出识别下述语言的 NFA, 并且符合规定的状态数。在所有问题中字母表均为 $\{0, 1\}$ 。

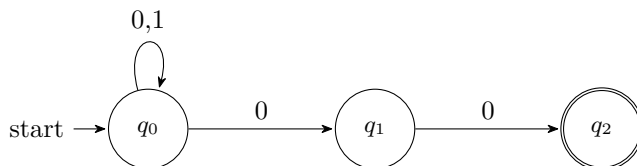
- ^A a. 语言 $\{w \mid w \text{ 以 } 00 \text{ 结束}\}$, 3 个状态。
- b. 练习 1.6c 中的语言, 5 个状态。
- c. 练习 1.61 中的语言, 6 个状态。
- d. 语言 $\{0\}$, 2 个状态。
- e. 语言 $0^* 1^* 0^+$, 3 个状态。
- ^A f. 语言 $1^* (001^+)^*$, 3 个状态。
- g. 语言 $\{\epsilon\}$, 1 个状态。
- h. 语言 0^* , 1 个状态。

图 13

方法总结: NFA 允许不完备的转移函数、对同一符号有多个去向、以及 ϵ 转移。构造时常用技巧: (1) 用起始态自环“等待”匹配时机, 由非确定性“猜”何时进入识别阶段; (2) 用 ϵ 转移实现并运算和 Kleene 星号; (3) 缺失的转移即“拒绝当前分支”。

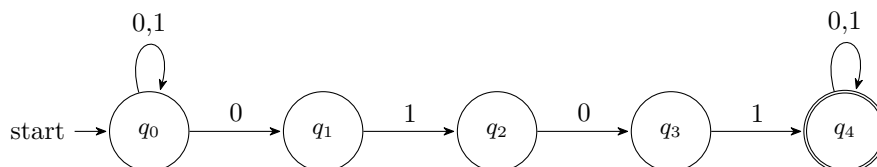
(a) $\{w \mid w \text{ 以 } 00 \text{ 结束}\}$, 3 个状态

q_0 靠自环消耗任意前缀, 非确定性地“猜”最后两个字符是 00 并分支到 q_1, q_2 。



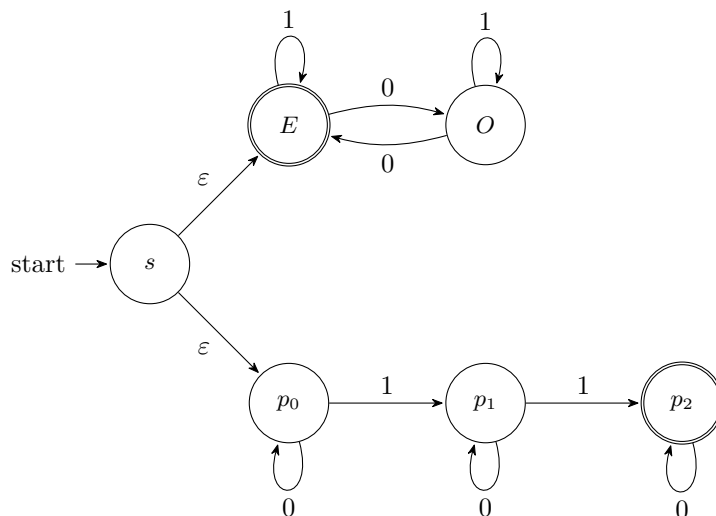
(b) $\{w \mid w \text{ 含子串 } 0101\}$, 5 个状态

与 1.6c 的 DFA 相比, NFA 无需设计失配回退—— q_0 自环“等”匹配时机。



(c) 练习 1.61 中的语言: 偶数个 0 或恰好 2 个 1, 6 个状态

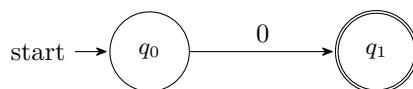
并运算用两条 ε 转移从起始态分叉到两个独立小 NFA。



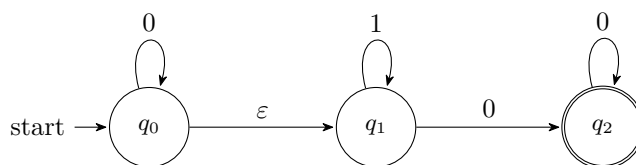
上分支: E = 已读偶数个 0 (接受), O = 奇数个 0。下分支: p_k = 已读 k 个 1, p_2 接受; 一旦再读 1 就无路可走 (被拒绝)。

(d) 语言 $\{0\}$, 2 个状态

只接受单字符 0。读 1 或超出 1 字符都无路可走。

(e) 语言 $0^*1^*0^+$, 3 个状态

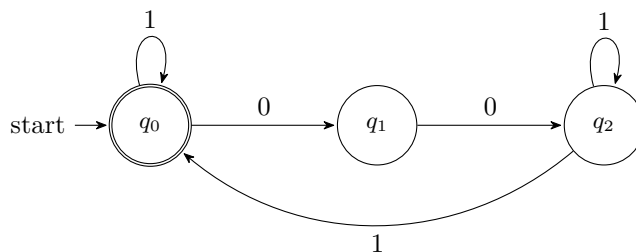
三个阶段对应三个状态。注意 0^+ 要求至少一个 0, 所以 q_2 必须 ** 读到 0 才能进入 ** (不能用 ε 跳进)。



q_0 消耗 0^* 前缀; ε 进入 q_1 消耗 1^* 中间; 读一个 0 进入 q_2 接受, q_2 再自环消耗剩余 0。

(f) 语言 $1^*(001^+)^*$, 3 个状态

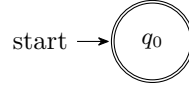
q_0 即接受态 (对应已完成一个合法串, 包括空串)。 q_0 自环吃掉任意多个 1; 读 '00' 经由 q_1 到 q_2 ; q_2 必须读至少一个 1 才能返回 q_0 , 从而保证每个 001 块末尾的 1^+ 至少有一个 1。



关键: q_2 只能通过“读 1”离开(到 q_0 或自身), 这保证了每个 ‘00’ 之后至少跟一个 1。NFA 的非确定性体现在: q_2 读到一个 1 时, 既可选择自环留在 q_2 (继续延长当前块的 1^+), 也可选择转移回 q_0 (结束当前块或整个串)。

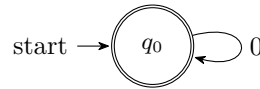
(g) 语言 $\{\varepsilon\}$, 1 个状态

只接受空串: 起始态即接受态, 无任何转移。



(h) 语言 0^* , 1 个状态

起始态即接受态, 加一个 0 自环。读到 1 无路可走。



1.11

^A 1.11 证明每一台 NFA 都能够被转换成只有一个接受状态的等价 NFA。

图 14

1.11 Let $N = (Q, \Sigma, \delta, q_0, F)$ be any NFA. Construct an NFA N' with a single accept state that recognizes the same language as N . Informally, N' is exactly like N except it has ε -transitions from the states corresponding to the accept states of N , to a new accept state, q_{accept} . State q_{accept} has no emerging transitions. More formally, $N' = (Q \cup \{q_{\text{accept}}\}, \Sigma, \delta', q_0, \{q_{\text{accept}}\})$, where for each $q \in Q$ and $a \in \Sigma_\varepsilon$

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{if } a \neq \varepsilon \text{ or } q \notin F \\ \delta(q, a) \cup \{q_{\text{accept}}\} & \text{if } a = \varepsilon \text{ and } q \in F \end{cases}$$

and $\delta'(q_{\text{accept}}, a) = \emptyset$ for each $a \in \Sigma_\varepsilon$.

图 15

案: 中文版此处翻译错误, δ' 的第二行中的两个条件应该为“且”

设 $N = (Q, \Sigma, \delta, q_0, F)$ 为任一 NFA。构造一个只有一个接受状态的 NFA N' , 它与 N 接受同样的语言。非形式地讲, N' 与 N 很相近, 但是 N' 从相对于 N 是接受状态的状态到新的接受状态 q_{accept} 有 ε 转移。状态 q_{accept} 上没有新的转移。形式地讲, $N' = (Q \cup \{q_{\text{accept}}\}, \Sigma, \delta', q_0, \{q_{\text{accept}}\})$, 其中对每一个 $q \in Q$ 和 $a \in \Sigma_\varepsilon$,

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{如果 } a \neq \varepsilon \text{ 或 } q \notin F \\ \delta(q, a) \cup \{q_{\text{accept}}\} & \text{如果 } a = \varepsilon \text{ 且 } q \in F \end{cases}$$

并且对每一个 $a \in \Sigma_\varepsilon$, $\delta'(q_{\text{accept}}, a) = \emptyset$ 。

图 16

1.13

1.13 令 F 是 $\{0, 1\}$ 上所有串构成的语言, 并且 F 中任意两个 1 之间间隔的符号数都不是奇数。画出识别 F 的 5 状态的 DFA 图。(先找一个 4 状态的 NFA 作 F 的补集可能有助于求解问题。)

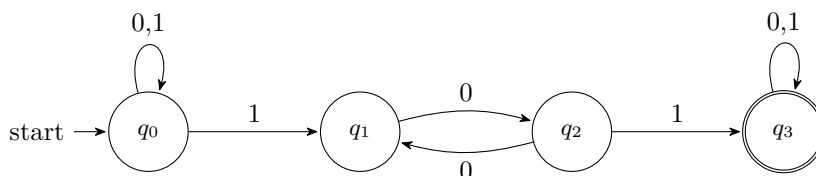
图 17

1.13 $F = \{w \mid w \text{ 中任意两个 } 1 \text{ 之间间隔的符号数都不是奇数}\}$

题意分析: 要求 F 中的串, 其中任意两个 1 之间的 0 的个数都是偶数 (包括 0 个)。没有 1 或只有一个 1 的串自动满足 (空真)。

策略: 按提示先构造识别 $\bar{F}$ 的 4 状态 NFA, 再用子集构造转为 DFA, 最后取补。

第一步: 识别 $\bar{F}$ 的 4 状态 NFA。 $\bar{F}$ = 存在某对 1, 它们之间的 0 的个数是奇数。NFA 利用非确定性猜违反的第一个 1, 然后验证其后读了奇数个 0 再读到 1。



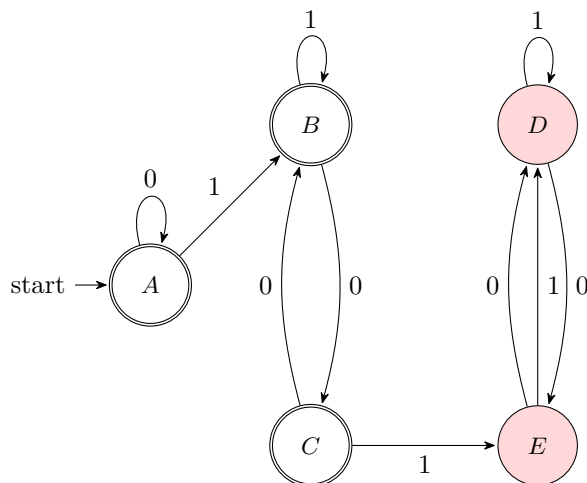
状态含义: q_0 = 读任意前缀; q_1 = 从猜到的第一个 1 起, 已读偶数个 0; q_2 = 已读奇数个 0; q_3 = 奇数个 0 后读到第二个 1, 违反对确认, 接受。

第二步: 子集构造转 DFA。

DFA 状态	读 0	读 1
$A = \{q_0\}$	$\{q_0\} = A$	$\{q_0, q_1\} = B$
$B = \{q_0, q_1\}$	$\{q_0, q_2\} = C$	$\{q_0, q_1\} = B$
$C = \{q_0, q_2\}$	$\{q_0, q_1\} = B$	$\{q_0, q_1, q_3\} = D$
$D = \{q_0, q_1, q_3\}$	$\{q_0, q_2, q_3\} = E$	$\{q_0, q_1, q_3\} = D$
$E = \{q_0, q_2, q_3\}$	$\{q_0, q_1, q_3\} = D$	$\{q_0, q_1, q_3\} = D$

得到 5 个可达状态。 $\bar{F}$ 的接受态是含 q_3 的状态: D, E 。

第三步: 取补得 F 的 5 状态 DFA。 将接受态翻转—— A, B, C 接受 (未含违反对), D, E 拒绝 (已确认违反)。



状态语义:

- A: 尚未读到 1(空串 ϵ 或全 0 串)
- B: 上个 1 之后读了偶数个 0(含 0 个)——若串此刻结束, 前面所有的 1-对间隔都合规
- C: 上个 1 之后读了奇数个 0 ——尚未确认违反 (若后面不再有 1 也可接受)
- D: 已经确认违反 (在 C 状态时读到过 1), 当前距最后的 1 读了偶数个 0
- E: 已经确认违反, 当前距最后的 1 读了奇数个 0

关键转移: $C \xrightarrow{1} E$ 是唯一“发生违反”的转移——在距上一个 1 奇数个 0 后又读到了一个 1。一旦进入 $\{D, E\}$ 就永远困在其中 (死区)。

1.14

- 1.14 a. 证明: 若 M 是一台识别语言 B 的 DFA, 交换 M 的接受状态与非接受状态得到一台新的 DFA, 则这台新 DFA 识别 B 的补集。因而, 正则语言类在补运算下封闭。
- b. 举例说明: 若 M 是一台识别语言 C 的 NFA, 交换 M 的接受状态与非接受状态, 得到一台新 NFA, 这台新 NFA 不一定识别 C 的补集。NFA 识别的语言类在补运算下封闭吗? 并解释回答。

图 18

解答

先证明对 DFA 成立, 再说明为什么对 NFA 直接交换接受状态与非接受状态并不正确。

DFA 情形. 设

$$M = (Q, \Sigma, \delta, q_0, F)$$

识别语言 B , 即

$$B = L(M) = \{w \in \Sigma^* \mid \delta^*(q_0, w) \in F\}.$$

构造新的 DFA

$$M' = (Q, \Sigma, \delta, q_0, Q \setminus F),$$

也就是只交换接受状态与非接受状态, 而保持转移函数不变。

对任意字符串 $w \in \Sigma^*$, 由于 DFA 的计算路径唯一,

$$w \in L(M') \iff \delta^*(q_0, w) \in Q \setminus F \iff \delta^*(q_0, w) \notin F.$$

另一方面,

$$\delta^*(q_0, w) \notin F \iff w \notin L(M) \iff w \notin B.$$

因此

$$L(M') = \Sigma^* \setminus B = \overline{B}.$$

所以, 只要把一个识别 B 的 DFA 的接受状态与非接受状态互换, 就得到一个识别 $\overline{B}$ 的 DFA。于是, 若 B 是正则语言, 则 $\overline{B}$ 也是正则语言。换言之, 正则语言类在补运算下封闭。

NFA 情形. 对于 NFA, 直接交换接受状态与非接受状态不一定得到补语言。下面给出一个反例。

设字母表

$$\Sigma = \{a\}.$$

构造 NFA

$$N = (Q, \Sigma, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1\}, \quad F = \{q_1\},$$

转移为

$$\delta(q_0, a) = \{q_0, q_1\}, \quad \delta(q_1, a) = \{q_1\}.$$

这个 NFA 识别的语言是

$$C = \{a^n \mid n \geq 1\} = a^+.$$

原因是: 只要输入非空串, 就可以在某一步从 q_0 转到 q_1 , 并最终停在接受状态。

现在交换接受状态与非接受状态, 得到

$$F' = Q \setminus F = \{q_0\}.$$

设新 NFA 为

$$N' = (Q, \Sigma, \delta, q_0, F').$$

由于对任意 a^n 都可以始终停留在状态 q_0 , 所以

$$L(N') = a^*.$$

但

$$\overline{C} = \Sigma^* \setminus a^+ = \{\varepsilon\}.$$

显然

$$L(N') = a^* \neq \{\varepsilon\} = \overline{C}.$$

因此, 对 NFA 直接交换接受状态与非接受状态, 并不一定得到原语言的补集。

原因分析. DFA 的接受基于唯一计算路径, 因此

$$w \in L(M) \iff \text{这唯一一条计算路径接受 } w.$$

所以把接受态与非接受态交换后，恰好就把“接受”变成“拒绝”，把“拒绝”变成“接受”。

但 NFA 的接受条件是

$$w \in L(N) \iff \text{存在一条计算路径接受 } w.$$

交换接受状态后，新 NFA 接受的是

存在一条计算路径停在原来的非接受状态。

而补语言要求的是

$$w \in \overline{L(N)} \iff \text{不存在任何一条计算路径接受 } w,$$

也就是

$\forall$ 计算路径，都不接受。

这里“存在一条路径”与“所有路径都不接受”并不等价，所以不能通过简单交换终态来取补。

结论. 虽然对 NFA 不能直接交换接受状态来取补，但 NFA 所识别的语言类仍然在补运算下封闭。因为任意 NFA 都可以先通过子集构造法转化为等价的 DFA，再对该 DFA 交换接受状态与非接受状态，即可得到补语言的识别器。■

1.15

1.15 给出一个反例，说明下述构造不能证明定理 1.24，即正则语言类在星号运算下封闭^①。

设 $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ 识别 A_1 。如下构造 $N = (Q, \Sigma, \delta, q_1, F)$ 。 N 应该识别 A_1^* 。

- N 的状态集是 N_1 的状态集。
- N 的起始状态与 N_1 的起始状态相同。
- $F = \{q_1\} \cup F_1$ 。

F 的接受状态是原来的接受状态加上它的起始状态。

- 定义 δ 如下：对每一个 $q \in Q$ 和每一个 $a \in \Sigma_\epsilon$ ，

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \notin F_1 \text{ 或 } a \neq \epsilon \\ \delta_1(q, a) \cup \{q_1\} & q \notin F_1 \text{ 或 } a = \epsilon \end{cases}$$

（建议：把这个构造转换成图，如图 1-26 所示。）

图 19

这题中文版的第二版和第三版均存在翻译错误，下图20 是英文版的题干

1.15 Give a counterexample to show that the following construction fails to prove Theorem 1.49, the closure of the class of regular languages under the star operation.⁷ Let $N_1 = (Q_1, \Sigma, \delta_1, q_1, F_1)$ recognize A_1 . Construct $N = (Q_1, \Sigma, \delta, q_1, F)$ as follows. N is supposed to recognize A_1^* .

- a. The states of N are the states of N_1 .
- b. The start state of N is the same as the start state of N_1 .
- c. $F = \{q_1\} \cup F_1$.
The accept states F are the old accept states plus its start state.
- d. Define δ so that for any $q \in Q_1$ and any $a \in \Sigma_\varepsilon$,

$$\delta(q, a) = \begin{cases} \delta_1(q, a) & q \notin F_1 \text{ or } a \neq \varepsilon \\ \delta_1(q, a) \cup \{q_1\} & q \in F_1 \text{ and } a = \varepsilon. \end{cases}$$

(Suggestion: Show this construction graphically, as in Figure 1.50.)

图 20

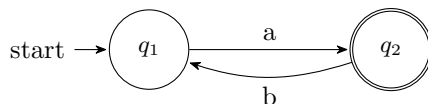
解答

这份证明的问题在于：所构造的新 NFA 并不一定识别 A_1^* 。下面给出一个反例。

反例的选择。取

$$A_1 = a(ba)^*.$$

令 N_1 是识别 A_1 的 NFA，其状态图如下：



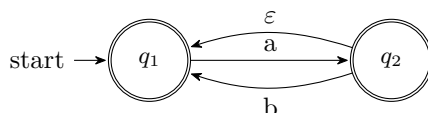
其中 q_1 是开始状态， q_2 是唯一接受状态。因此

$$L(N_1) = \{a(ba)^k \mid k \geq 0\} = a(ba)^* = A_1.$$

按题中错误方法构造新 NFA。按照题目中的构造，新 NFA N 的接受状态变为

$$F = \{q_1\} \cup F_1 = \{q_1, q_2\},$$

并且因为 $q_2 \in F_1$ ，所以还要加入一条 ε -转移 $q_2 \rightarrow q_1$ 。于是得到下图：



说明该构造出错。在这个新 NFA 中，

$$q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_1,$$

而 q_1 已经是接受状态，所以

$$ab \in L(N).$$

但是

$$ab \notin A_1^*.$$

这是因为 $A_1 = a(ba)^*$ 中的每个非空字符串都以 a 开头并以 a 结尾，因此 A_1^* 中每个非空块也都以 a 结尾，从而任意非空拼接结果仍以 a 结尾。字符串 ab 以 b 结尾，所以不可能属于 A_1^* 。

因此

$$L(N) \neq A_1^*.$$

这就说明题目中的构造不能用来证明正则语言类对 Kleene 星号运算封闭。

错误的本质. 问题出在它直接把原开始状态 q_1 设成了接受状态。这样一来，只要某条路径在“尚未完整读完一个 A_1 -块”时返回到了 q_1 ，机器就可能提前接受，从而识别出不属于 A_1^* 的字符串。

正确的做法是：新建一个**新的开始接受状态**，并从它向旧开始状态加一条 ε -边；同时从每个旧接受状态向旧开始状态加 ε -边。这样才能正确识别 A_1^* 。■

1.17

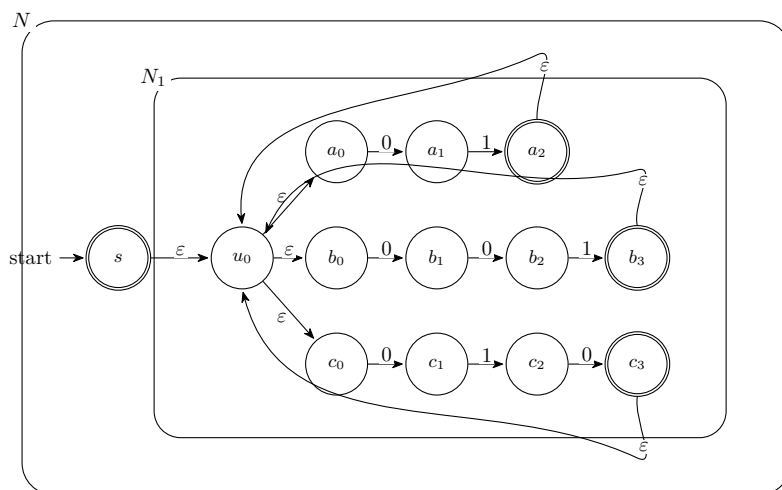
1.17 a. 给出识别语言 $(01 \cup 001 \cup 010)^*$ 的 NFA。

b. 将给出的 NFA 转换成等价的 DFA。给出在该 DFA 开始状态可达的部分即可。

图 21

解答

识别 $(01 \cup 001 \cup 010)^*$ 的 NFA. 按照正则表达式到 NFA 的标准构造，先分别构造识别 01、001、010 的三个小 NFA，再用 ε 边做并运算得到 N_1 ，最后在外层按星号构造得到 N 。



其中 N_1 识别 $01 \cup 001 \cup 010$ ，上、中、下三条分支分别识别 01、001、010。外层 N 的新开始状态 s 同时是接受状态，因此接受空串；从 s 到 u_0 的 ε 边进入 N_1 ，从 N_1 的接受状态回到 u_0 的 ε 边允许重复读下一个块。

子集构造得到的等价 DFA. 为了得到较小的 DFA，可以先把上面的构造 NFA 化简为只记录“当前匹配到块内哪个前缀”的四状态 NFA。设该 NFA 的状态为 q_0, q_1, q_2, q_3 ，其中 q_0 是开始状态和接受状态，并有如下含义：

状态	含义
q_0	当前正好停在块边界，可以接受；
q_1	已经读到某个块的前缀 0；
q_2	已经读到某个块的前缀 00；
q_3	已经读到某个块的前缀 01，还可继续读 0 形成 010。

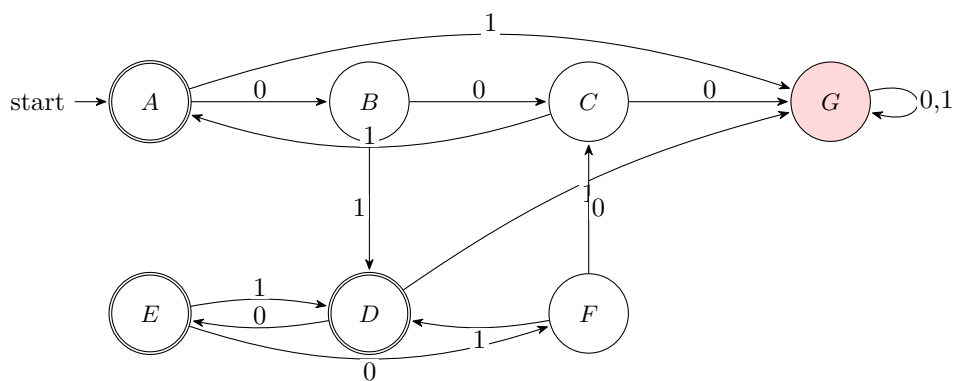
对这个 NFA 做子集构造时，DFA 的每个状态都是一个 NFA 状态集合。只保留从开始集合 $\{q_0\}$ 出发能够到达的集合，得到：

DFA 状态	NFA 状态集合	为什么会出现
A	$\{q_0\}$	开始集合
B	$\{q_1\}$	A 读 0
C	$\{q_2\}$	B 读 0
D	$\{q_0, q_3\}$	B 或 F 读 1
E	$\{q_0, q_1\}$	D 读 0
F	$\{q_1, q_2\}$	E 读 0
G	$\emptyset$	无合法转移时进入的死状态

它们之间的转移由逐点取并得到：

状态	0	1
A	B	G
B	C	D
C	G	A
D	E	G
E	F	D
F	C	D
G	G	G

因为原 NFA 的唯一接受态是 q_0 ，所以 DFA 中所有包含 q_0 的状态都是接受状态，即 A, D, E ； $G = \emptyset$ 是死状态。对应的 DFA 如下。



1.19

1.19 对于下述每一个语言，给出 4 个字符串，其中 2 个是这个语言的成员，2 个不是这个语言

图 22

的成员。这里假设字母表 $\Sigma = \{a, b\}$ 。

- | | |
|-------------------|--|
| a. a^*b^* | e. $\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$ |
| b. $a(ba)^*b$ | f. $aba \cup bab$ |
| c. $a^* \cup b^*$ | g. $(\varepsilon \cup a)b$ |
| d. $(aaa)^*$ | h. $(a \cup ba \cup bb)\Sigma^*$ |

图 23

解答

下面给出每个语言的两个成员和两个非成员。这里约定字母表为 $\Sigma = \{a, b\}$ 。

题号	语言	属于该语言的字符串	不属于该语言的字符串
(a)	a^*b^*	$\varepsilon, aaabbb$	$ba, abab$
(b)	$a(ba)^*b$	$ab, abab$	ε, aab
(c)	$a^* \cup b^*$	aaa, bb	ab, ba
(d)	$(aaa)^*$	$\varepsilon, aaaaaa$	a, aa
(e)	$\Sigma^*a\Sigma^*b\Sigma^*a\Sigma^*$	$aba, baaba$	ε, aab
(f)	$aba \cup bab$	aba, bab	ε, ab
(g)	$(\varepsilon \cup a)b$	b, ab	ε, a
(h)	$(a \cup ba \cup bb)\Sigma^*$	a, ba	ε, b

其中 (e) 要求字符串中存在一个按顺序出现的子序列 a, b, a ；例如 $baaba$ 中第 2, 4, 5 个字符可取为 a, b, a 。而 (h) 表示字符串必须以 a 、 ba 或 bb 开头，所以空串和单独的 b 都不属于该语言。

1.30

- 1.30** 指出下述关于 0^*1^* 不是正则语言的“证明”的错误(由于 0^*1^* 是正则语言所以“证明”存在错误)：用反证法证明。假设 0^*1^* 是正则的。设 p 是泵引理给出的关于 0^*1^* 的泵长度。选择 s 为串 0^p1^p 。已知 s 是 0^*1^* 的一个成员，但例 1.38 表明 s 不能被抽取。这样得到矛盾。因此 0^*1^* 不是正则的。

图 24

解答

该证明的错误在于混淆了两个不同的语言。语言

$$0^*1^* = \{0^m1^n \mid m, n \geq 0\}$$

本身由正则表达式 0^*1^* 描述，因而是正则语言，不可能由泵引理推出其非正则性。

错误发生在把例 1.38 中关于

$$B = \{0^n 1^n \mid n \geq 0\}$$

的结论用于 0^*1^* 。对于 B ，字符串 $0^p 1^p$ 不能被泵，是因为泵动会破坏 0 与 1 的数量相等；但 0^*1^* 并不要求二者数量相等，只要求所有 0 在所有 1 之前。

事实上，取

$$s = 0^p 1^p \in 0^*1^*,$$

可作分解

$$x = \varepsilon, \quad y = 0, \quad z = 0^{p-1} 1^p.$$

于是对任意 $i \geq 0$,

$$xy^i z = 0^{p-1+i} 1^p \in 0^*1^*.$$

因此该字符串在语言 0^*1^* 中是可以被泵的。

泵引理用于证明非正则时，必须证明：存在某个选定的 $s \in A$ ，使得对所有满足条件的分解 $s = xyz$ ，都能找到 $i \geq 0$ 令 $xy^i z \notin A$ 。题中证明只借用了 $B = \{0^n 1^n \mid n \geq 0\}$ 的结论，而该结论不能迁移到约束不同的语言 0^*1^* 。■

方法整理

证明一个语言正则，通常采用构造法。也就是说，给出下列任意一种等价描述即可：

$$\text{正则表达式} \iff \text{NFA} \iff \text{DFA}.$$

因此，若能直接写出正则表达式，或构造出识别该语言的 NFA/DFA，就已经证明了该语言正则。例如 0^*1^* 由正则表达式本身描述，所以正则。

另一种常用方法是使用正则语言的闭包性质。已知正则语言类在并、连接、星号、补、交、差、逆、同态和逆同态等运算下封闭。因此，如果目标语言可由已知正则语言经过这些运算得到，也可推出其正则性。

证明一个语言非正则，则通常用反证法：先假设该语言正则，再推出矛盾。常用工具有：

- (1) 泵引理：设语言正则，取足够长且结构受限的字符串 s ，证明对任意合法分解 $s = xyz$ ，总能选择某个 i 使 $xy^i z$ 离开该语言。
- (2) 闭包性质反证：若假设 A 正则，则通过与某个正则语言求交、取差、同态等操作得到一个已知非正则语言，从而矛盾。
- (3) 区分串方法：构造无限多个两两可区分的前缀，说明任何 DFA 都需要无限多个状态，因此语言非正则。

需要注意，泵引理给出的是正则语言的必要条件，而不是充分条件。其逻辑形式是

$$A \text{ 正则} \implies A \text{ 满足泵引理}.$$

因此可以使用逆否命题

$$A \text{ 不满足泵引理} \implies A \text{ 非正则},$$

但不能反过来认为

$$A \text{ 满足泵引理} \implies A \text{ 正则}.$$

所以，泵引理是证明非正则的工具，而不是证明正则的工具。证明正则时应给出 RE、NFA、DFA，或把语言化归为正则语言闭包运算的结果。

1.32

1.32 设

$$\Sigma_3 = \left\{ \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \dots, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \right\}$$

Σ_3 包含所有高度为 3 的 0 和 1 的列。 Σ_3 上的字符串给出三行 0 和 1。把每一行看作一个二进制数，令

$$B = \{w \in \Sigma_3^* \mid w \text{ 最下面的一行等于上面两行的和}\}$$

例如，

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix} \in B, \text{ 但是 } \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \notin B$$

证明 B 是正则的(提示: 证明 B^R 是正则的会容易一些。假设已知问题 1.31 中的结果)。

图 25

证明

二进制加法应从最低位向最高位检查。故先证明反转语言 B^R 正则，再由正则语言对反转封闭推出 B 正则。

设输入字母为三位列

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}, \quad a, b, c \in \{0, 1\},$$

分别表示两个加数与和的对应二进制位。在 B^R 中，读入顺序正是从最低位到最高位，所以只需记录当前进位。

构造 DFA

$$M = (Q, \Sigma_3, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1, q_d\}, \quad F = \{q_0\}.$$

状态 q_0, q_1 分别表示当前进位为 0, 1, 而 q_d 为死状态。对 $r \in \{0, 1\}$ 与任意列字母

$$\sigma = \begin{bmatrix} a \\ b \\ c \end{bmatrix},$$

定义转移如下: 若

$$c \equiv a + b + r \pmod{2},$$

则

$$\delta(q_r, \sigma) = q_{\lfloor (a+b+r)/2 \rfloor};$$

否则令 $\delta(q_r, \sigma) = q_d$ 。并且对一切 $\sigma \in \Sigma_3$,

$$\delta(q_d, \sigma) = q_d.$$

该 DFA 恰好逐位验证二进制加法，并把进位传递到下一列。输入结束时接受当且仅当最终进位为 0，即当前状态为 q_0 。因此 B^R 是正则语言。由于正则语言对反转封闭，

$$B = (B^R)^R$$

也是正则语言。■

解释. 这个证明的核心是找出二进制加法检查中真正需要记住的信息。若从右向左读，即从最低位向最高位读，那么已经读过的低位对后续检查的影响只体现为当前进位。进位只有两种可能，0 或 1，所以这是有限信息，可以由 DFA 的有限状态记录。

具体地，DFA 处在 q_r 时表示“到目前为止读过的低位都合法，且传给下一位的进位为 r ”。读入一列

$$\begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

后，DFA 检查 c 是否等于 $a + b + r$ 的末位。如果不是，就进入死状态；如果是，就把状态更新为新的进位

$$\left\lfloor \frac{a + b + r}{2} \right\rfloor.$$

因此 DFA 并不需要记住已经读过的全部数字，只需要记住一个有限状态：进位是否为 1。这正是该语言正则的原因。

1.34

1.34 设 Σ_2 与问题 1.33 中的相同。把每一行看作是一个二进制数，并且设

$$D = \{w \in \Sigma_2^* \mid w \text{ 上一行表示的数大于下一行表示的数}\}$$

例如，

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \in D, \quad \text{但是} \quad \begin{bmatrix} 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} \notin D$$

证明 D 是正则的。

图 26

证明

设字母表为

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

字符串

$$w = \begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \begin{bmatrix} a_2 \\ b_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix}$$

表示两行二进制数 $a_1 a_2 \cdots a_n$ 与 $b_1 b_2 \cdots b_n$ 。从左向右比较二者时，大小关系由最左边第一个不同位决定。

构造 DFA

$$M = (Q, \Sigma_2, \delta, q_-, F),$$

其中

$$Q = \{q_-, q_+, q_-\}, \quad F = \{q_+\}.$$

状态 q_- 表示目前为止两行相等, q_+ 表示已经确定上面一行较大, q_- 表示已经确定上面一行较小。

在状态 q_- 时, 定义

$$\begin{aligned} \delta\left(q_-, \begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) &= q_-, & \delta\left(q_-, \begin{bmatrix} 1 \\ 1 \end{bmatrix}\right) &= q_-, \\ \delta\left(q_-, \begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) &= q_+, & \delta\left(q_-, \begin{bmatrix} 0 \\ 1 \end{bmatrix}\right) &= q_-. \end{aligned}$$

一旦已经确定大小关系, 后续位不再改变结果。因此对任意 $\sigma \in \Sigma_2$,

$$\delta(q_+, \sigma) = q_+, \quad \delta(q_-, \sigma) = q_-.$$

该 DFA 恰好记录从左至右比较时所需的有限信息: 尚未分出大小、已经上大于下、已经上小于下。最终接受当且仅当最左边第一个不同位为

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix},$$

即上面一行表示的二进制数大于下面一行。因此 $L(M) = D$, 故 D 是正则语言。■

1.35

1.35 设 Σ_2 与问题 1.33 中的相同。把上、下行都看作是 0 和 1 的字符串, 并且设

$$E = \{w \in \Sigma_2^* \mid w \text{ 的下一行是上一行的反转}\}$$

证明 E 不是正则的。

图 27

证明

设

$$\Sigma_2 = \left\{ \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right\}.$$

若

$$w = \begin{bmatrix} a_1 \\ b_1 \end{bmatrix} \begin{bmatrix} a_2 \\ b_2 \end{bmatrix} \cdots \begin{bmatrix} a_n \\ b_n \end{bmatrix},$$

则 $w \in E$ 当且仅当

$$b_1 b_2 \cdots b_n = a_n a_{n-1} \cdots a_1.$$

用泵引理反证。假设 E 正则, 令 p 为其泵长度。取

$$s = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^p \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p.$$

其上行是 $0^p 1^p$, 下行是 $1^p 0^p = (0^p 1^p)^R$, 故 $s \in E$, 且 $|s| = 2p \geq p$ 。

由泵引理, 存在分解 $s = xyz$, 满足

$$|xy| \leq p, \quad |y| > 0,$$

并且对所有 $i \geq 0$, 有 $xy^iz \in E$ 。由于 $|xy| \leq p$, 子串 y 完全位于前 p 个字母中, 所以

$$y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k \quad (k > 0).$$

令 $i = 0$, 得到

$$xz = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^{p-k} \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p.$$

此时上行是 $0^{p-k}1^p$, 下行是 $1^{p-k}0^p$ 。然而

$$(0^{p-k}1^p)^R = 1^p0^{p-k} \neq 1^{p-k}0^p.$$

故 $xz \notin E$, 与泵引理要求 $xy^0z \in E$ 矛盾。因此 E 不是正则语言。■

解释. 该语言要求下行等于上行的反转。也就是说, 自动机读到前面的上行信息后, 必须在后面以相反顺序逐位比较。随着输入长度增加, 需要保存的前缀可以任意长; 而 DFA 只有有限多个状态, 不能保存任意长的前缀信息。这就是 E 非正则的本质原因。

这里并不是只考察了某一种特殊分解。泵引理要求对任意满足 $|xy| \leq p$ 、 $|y| > 0$ 的分解都能泵。我们选取的

$$s = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^p \begin{bmatrix} 1 \\ 0 \end{bmatrix}^p$$

有一个关键性质: 它的前 p 个符号全都是

$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

因此无论合法分解 $s = xyz$ 怎样选, 只要 $|xy| \leq p$ 且 $|y| > 0$, 子串 y 都必然具有形式

$$y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k \quad (k > 0).$$

所以证明中令 $y = \begin{bmatrix} 0 \\ 1 \end{bmatrix}^k$ 并不是任取一种情况, 而是概括了所有合法分解。随后取 $i = 0$, 对任意这样的 $k > 0$ 都会破坏上下两行的反转关系, 故得到的矛盾满足泵引理反证所需的量词结构。

1.36

1.36 设 $B_n = \{a^k \mid k \text{ 是 } n \text{ 的整数倍}\}$ 。证明: 对每一个 $n \geq 1$, 语言 B_n 是正则的。

图 28

证明

固定任意 $n \geq 1$ 。构造 DFA

$$M_n = (Q, \{a\}, \delta, q_0, F),$$

其中

$$Q = \{q_0, q_1, \dots, q_{n-1}\}, \quad F = \{q_0\}.$$

状态 q_i 表示目前已经读入的 a 的个数模 n 余 i 。转移函数定义为

$$\delta(q_i, a) = q_{i+1 \bmod n}, \quad 0 \leq i \leq n-1.$$

于是对任意 $k \geq 0$,

$$\delta^*(q_0, a^k) = q_{k \bmod n}.$$

因此

$$a^k \in L(M_n) \iff \delta^*(q_0, a^k) = q_0 \iff k \equiv 0 \pmod{n}.$$

也就是说,

$$L(M_n) = \{a^k \mid k \text{ 是 } n \text{ 的整数倍}\} = B_n.$$

所以对每个 $n \geq 1$, B_n 都由一个有限状态自动机识别, 故 B_n 是正则语言。■

解释. 这里 DFA 需要记住的不是完整的长度 k , 而只是 k 除以 n 的余数。余数只有

$$0, 1, \dots, n-1$$

这 n 种可能, 因此可以用有限多个状态记录。每读入一个 a , 余数加 1 并对 n 取模; 最终回到余数 0 的状态, 正好表示长度是 n 的整数倍。

1.40

1.40 如果存在字符串 z 使得 $xz=y$, 则称字符串 x 是字符串 y 的**前缀**(prefix)。如果 x 是 y 的前缀且 $x \neq y$, 则称 x 是 y 的**真前缀**(proper prefix)。下面每小题目定义一个语言 A 上的运算。证明: 正则语言类在每个运算下封闭。

- ^A a. $\text{NOPREFIX}(A) = \{w \in A \mid w \text{ 任一真前缀都不是 } A \text{ 的元素}\}$
 b. $\text{NOEXTEND}(A) = \{w \in A \mid w \text{ 不是 } A \text{ 中任何字符串的真前缀}\}$

图 29

证明

设 A 是正则语言, 令

$$M = (Q, \Sigma, \delta, q_0, F)$$

为识别 A 的 DFA。

(a) 先证明 $\text{NOPREFIX}(A)$ 正则。

构造 DFA

$$M_1 = (Q \cup \{q_d\}, \Sigma, \delta_1, q_0, F),$$

其中 q_d 是死状态。定义

$$\delta_1(q_d, a) = q_d \quad (a \in \Sigma),$$

且对 $q \in Q$,

$$\delta_1(q, a) = \begin{cases} q_d, & q \in F, \\ \delta(q, a), & q \notin F. \end{cases}$$

这个构造的含义是：一旦读到某个已经属于 A 的前缀，而输入还没有结束，则该前缀就是真前缀，于是进入死状态。只有当第一次到达接受状态恰好发生在输入结束时，才接受。因此

$$L(M_1) = \text{NOPREFIX}(A),$$

所以 $\text{NOPREFIX}(A)$ 正则。

(b) 再证明 $\text{NOEXTEND}(A)$ 正则。

关键是筛掉那些“还能继续走到接受态”的状态。把 M 看成一个有限有向图：状态是顶点，每条转移边

$$q \xrightarrow{a} \delta(q, a)$$

是一条带标号的有向边。定义

$$P = \{q \in Q \mid \text{从 } q \text{ 出发存在一条长度至少为 } 1 \text{ 的路径通向 } F\}.$$

等价地，

$$P = \{q \in Q \mid \exists x \in \Sigma^+, \delta^*(q, x) \in F\}.$$

这里并不是枚举无穷多个字符串 x ，而是在有限状态图上做可达性判断。

具体地，先在反向图中从所有接受状态 F 出发做 BFS 或 DFS，得到

$$R = \{q \in Q \mid \text{从 } q \text{ 出发可以经零步或多步到达 } F\}.$$

然后取

$$P = \{q \in Q \mid \exists a \in \Sigma, \delta(q, a) \in R\}.$$

这样正好保证从 q 到接受态至少走了一步；特别地，若某个接受状态有自环或能绕一圈回到接受状态，它也会被放入 P 。

构造 DFA

$$M_2 = (Q, \Sigma, \delta, q_0, F'), \quad F' = F \setminus P.$$

也就是说，状态、字母表、转移函数、开始状态全部不变，只把接受状态改为 $F \setminus P$ 。

若 w 被 M_2 接受，设

$$r = \delta^*(q_0, w).$$

则 $r \in F' = F \setminus P$ 。所以 $r \in F$ ，从而 $w \in A$ ；同时 $r \notin P$ ，说明从 r 出发不可能再经非空路径到达任何接受状态。于是不存在非空字符串 z 使得 $wz \in A$ ，故 w 不是 A 中任何字符串的真前缀。

反过来，若 $w \in A$ 且 w 不是 A 中任何字符串的真前缀，令 $r = \delta^*(q_0, w)$ 。由于 $w \in A$ ，有 $r \in F$ ；又因为 w 不能继续扩展成 A 中的字符串，从 r 出发不存在通向接受状态的非空路径，所以 $r \notin P$ 。于是 $r \in F \setminus P = F'$ ，故 M_2 接受 w 。

因此

$$L(M_2) = \text{NOEXTEND}(A),$$

所以 $\text{NOEXTEND}(A)$ 正则。

综上，正则语言类在 NOPREFIX 与 NOEXTEND 运算下封闭。■

解释。 $\text{NOPREFIX}(A)$ 要求“在接受当前串之前，不能已经接受过某个真前缀”，所以 DFA 只需在读入过程中检查是否提前进入过接受状态；若提前接受后还有输入，就进入死状态。

这里使用的 δ^* 是 DFA 转移函数 δ 的扩展。若

$$\delta : Q \times \Sigma \rightarrow Q$$

表示读入一个字符后的状态转移, 则

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

表示从某个状态出发, 读完整个字符串后到达的状态。它由

$$\delta^*(q, \varepsilon) = q, \quad \delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

递归定义, 其中 $w \in \Sigma^*$, $a \in \Sigma$ 。因此 $\delta^*(q, z) \in F$ 的意思是: 从状态 q 出发读完整个字符串 z 后到达接受状态。这是自动机理论中的标准记号。

NOEXTEND(A) 要求“当前串虽然在 A 中, 但不能继续扩展成另一个 A 中的串”。从改 DFA 的角度看, 就是检查每个接受状态后面是否还存在通往某个接受状态的非空路径: 若存在, 就说明停在该状态的字符串还能继续扩展成 A 中更长的字符串, 因此这个状态不能保留为接受状态; 若不存在, 才保留它。这个过程完全发生在有限状态图上, 所以是有限的构造。

什么叫封闭。一般地, 称一类语言 $\mathcal{C}$ 在某个运算 T 下封闭, 是指对 $\mathcal{C}$ 中的语言施行该运算后, 所得结果仍属于 $\mathcal{C}$ 。若 T 是一元运算, 则要求

$$A \in \mathcal{C} \implies T(A) \in \mathcal{C};$$

若 T 是二元运算, 则要求

$$A, B \in \mathcal{C} \implies T(A, B) \in \mathcal{C}.$$

在本章里, $\mathcal{C}$ 就是正则语言类。因此 1.40 的结论是: 若 A 正则, 则 NOPREFIX(A) 与 NOEXTEND(A) 仍正则。证明这类封闭性命题的常见方法, 就是从识别原语言的 DFA 或 NFA 出发, 直接构造识别新语言的自动机。

1.42

1.42 对语言 A 和语言 B , 设 A 和 B 的完全间隔交叉(perfect shuffle)为:

$$\{w \mid w = a_1 b_1 \cdots a_k b_k, \text{ 其中 } a_1 \cdots a_k \in A \text{ 并且 } b_1 \cdots b_k \in B, \text{ 任一 } a_i, b_i \in \Sigma\}$$

证明: 正则语言类在完全间隔交叉下封闭。

图 30

证明

设 $A, B \subseteq \Sigma^*$ 都是正则语言。取识别它们的 DFA

$$M_A = (Q_A, \Sigma, \delta_A, q_A, F_A), \quad M_B = (Q_B, \Sigma, \delta_B, q_B, F_B).$$

我们构造 DFA

$$M = (Q_A \times Q_B \times \{0, 1\}, \Sigma, \delta, (q_A, q_B, 0), F),$$

其中第三个分量用来记录当前读入的位置奇偶:

$$0: \text{下一位应送入 } M_A, \quad 1: \text{下一位应送入 } M_B.$$

接受状态取为

$$F = F_A \times F_B \times \{0\}.$$

这表示输入已经恰好读完若干对符号，并且奇数位形成的串被 M_A 接受，偶数位形成的串被 M_B 接受。

转移函数定义为

$$\delta((p, q, 0), a) = (\delta_A(p, a), q, 1), \quad \delta((p, q, 1), b) = (p, \delta_B(q, b), 0),$$

其中 $p \in Q_A$, $q \in Q_B$, $a, b \in \Sigma$ 。

下面证明 $L(M)$ 正是 A 与 B 的完全间隔交叉。

若

$$w = a_1 b_1 a_2 b_2 \cdots a_k b_k$$

且

$$a_1 a_2 \cdots a_k \in A, \quad b_1 b_2 \cdots b_k \in B,$$

则 M 读入奇数位时只更新第一个分量，读入偶数位时只更新第二个分量。因此读完整个 w 后到达状态

$$(\delta_A^*(q_A, a_1 a_2 \cdots a_k), \delta_B^*(q_B, b_1 b_2 \cdots b_k), 0).$$

由于两行分别属于 A 与 B ，前两个分量落在 F_A 与 F_B 中；又因为 w 长度为偶数，第三个分量回到 0。故 $w \in L(M)$ 。

反过来，若 $w \in L(M)$ ，则第三个分量最终为 0，所以 w 的长度必为偶数。于是可唯一写成

$$w = a_1 b_1 a_2 b_2 \cdots a_k b_k.$$

设

$$x = a_1 a_2 \cdots a_k, \quad y = b_1 b_2 \cdots b_k.$$

根据转移定义， M 读完 w 后的前两个分量分别正是

$$\delta_A^*(q_A, x), \quad \delta_B^*(q_B, y).$$

又因为 w 被接受，这两个状态分别属于 F_A 与 F_B ，故

$$x \in A, \quad y \in B.$$

因此 w 正是由 $x \in A$ 与 $y \in B$ 按完全间隔交叉得到的字符串。

综上，

$$L(M) = \{w \mid w \text{ 是 } A \text{ 与 } B \text{ 的完全间隔交叉}\}.$$

所以正则语言类在完全间隔交叉运算下封闭。■

解释. 这个构造的核心很直接：DFA 一边读输入，一边把奇数位送给识别 A 的自动机，把偶数位送给识别 B 的自动机；额外的一个二值状态只负责记录“当前该轮到哪一边读”。由于需要记住的信息只有两个分自动机的当前状态以及当前位置的奇偶性，所以总状态数仍然是有限的。

就 1.42 而言，这里讨论的运算是“完全间隔交叉”：输入是两个语言 A, B ，输出是由 A 中字符串与 B 中字符串按奇偶位交错得到的所有字符串组成的语言。所谓“正则语言类在这个运算下封闭”，意思是：只要 A 与 B 都正则，那么它们的完全间隔交叉所得语言也仍然正则。上面的乘积 DFA 就给出了这一点的构造性证明。

1.52

1.52 Myhill-Nerode 定理。参见问题 1.51, 设 L 是一个语言, X 是一个字符串集合。如果 X 中的任意两个不同的字符串都是用 L 可区分的, 则称 X 是用 L 两两可区分的。定义 L 的指数为用 L 两两可区分的集合中的元素个数的最大值。 L 的指数可能是有穷的或无穷的。

- 证明: 如果 L 被一台有 k 个状态的 DFA 识别, 则 L 的指数不超过 k 。
- 证明: 如果 L 的指数是一个有穷数 k , 则它被一台有 k 个状态的 DFA 识别。
- 由此得到: L 是正则的当且仅当它有有穷的指数。而且, 它的指数是识别它的最小 DFA 的大小。

图 31

证明

记

$$x \equiv_L y$$

表示字符串 $x, y \in \Sigma^*$ 用 L 不可区分, 即

$$\forall z \in \Sigma^*, \quad xz \in L \iff yz \in L.$$

若存在某个 $z \in \Sigma^*$ 使得

$$xz \in L, \quad yz \notin L$$

或反过来, 则称 x, y 可由 L 区分。

(a) 设 L 被一台有 k 个状态的 DFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

识别, 并且 $|Q| = k$ 。任取一个用 L 两两可区分的集合

$$X \subseteq \Sigma^*.$$

对每个 $x \in X$, 看它从初态出发到达的状态 $\delta^*(q_0, x)$ 。若存在不同的 $x, y \in X$ 满足

$$\delta^*(q_0, x) = \delta^*(q_0, y),$$

则对任意 $z \in \Sigma^*$, 都有

$$\delta^*(q_0, xz) = \delta^*(\delta^*(q_0, x), z) = \delta^*(\delta^*(q_0, y), z) = \delta^*(q_0, yz).$$

因而

$$xz \in L \iff yz \in L.$$

这说明 x, y 用 L 不可区分, 与 X 中任意两个不同字符串都应可区分矛盾。

因此, X 中不同字符串必须到达不同状态。于是映射

$$x \mapsto \delta^*(q_0, x)$$

从 X 到 Q 是单射, 所以

$$|X| \leq |Q| = k.$$

故 L 的指数不超过 k 。

- (b) 设 L 的指数有限, 且等于 k 。因为 L 的指数为 k , 所以关系 $\equiv_L$ 将 Σ^* 分成 k 个等价类。以这些等价类为状态, 构造 DFA

$$M = (Q, \Sigma, \delta, q_0, F).$$

令

$$Q = \{[x] \mid x \in \Sigma^*\},$$

其中

$$[x] = \{y \in \Sigma^* \mid y \equiv_L x\}.$$

因为指数是 k , 所以 $|Q| = k$ 。

初始状态定义为

$$q_0 = [\varepsilon].$$

转移函数定义为

$$\delta([x], a) = [xa].$$

需要说明它是良定义的。若

$$[x] = [y],$$

即 $x \equiv_L y$, 则对任意 $z \in \Sigma^*$, 因为 $x \equiv_L y$ 对字符串 az 也成立, 所以

$$xaz \in L \iff yaz \in L.$$

也就是

$$(xa)z \in L \iff (ya)z \in L.$$

因此 $xa \equiv_L ya$, 从而

$$[xa] = [ya].$$

所以转移函数定义良好。

接受状态定义为

$$F = \{[x] \mid x \in L\}.$$

这同样是良定义的。若 $[x] = [y]$, 即 $x \equiv_L y$, 取 $z = \varepsilon$, 则

$$x \in L \iff y \in L.$$

因而同一个等价类中的字符串要么全在 L 中, 要么全不在 L 中, 所以接受状态集合定义良好。

下面证明该 DFA 识别 L 。对任意字符串

$$w = a_1 a_2 \cdots a_n,$$

由转移定义可归纳得到

$$\delta^*([\varepsilon], w) = [w].$$

因此

$$w \in L(M) \iff \delta^*([\varepsilon], w) \in F \iff [w] \in F \iff w \in L.$$

所以

$$L(M) = L.$$

因而 L 被一台有 k 个状态的 DFA 识别。

(c) 由 (a) 可知, 若 L 被某台 k 状态 DFA 识别, 则 L 的指数不超过 k 。因此, 若 L 是正则语言, 则存在有限状态 DFA 识别它, 从而 L 的指数有限。

由 (b) 可知, 若 L 的指数有限且等于 k , 则可构造一台有 k 个状态的 DFA 识别 L 。所以

$$L \text{ 正则} \iff L \text{ 的指数有限.}$$

此外, 因为任何识别 L 的 DFA 的状态数都至少是 L 的指数, 而我们又能构造一台恰有指数个状态数的 DFA, 所以

$$L \text{ 的指数} = \text{识别 } L \text{ 的最小 DFA 的状态数.}$$

■

解释. Myhill–Nerode 定理的核心是: DFA 的一个状态只代表“从这里继续往后读, 未来会发生什么”。若两个前缀 x, y 对所有后缀 z 都表现完全一样, 即

$$xz \in L \iff yz \in L,$$

那么自动机没有必要把它们分开; 反之, 只要存在某个后缀能把它们区分开, 自动机就必须给它们不同的状态。因此, 最小 DFA 的状态本质上就是这些“未来行为”不同的等价类。

1.53

设 $\Sigma = \{0, 1, +, =\}$ 以及

$$ADD = \{x = y + z \mid x, y, z \text{ 是二进制整数, 并且 } x \text{ 是 } y \text{ 与 } z \text{ 的和}\}$$

图 32

证明

设

$$ADD = \{x = y + z \mid x, y, z \text{ 是二进制整数, 且 } x = y + z\}.$$

我们用闭包性质反证。令

$$R = \{10^i = 10^j + 0 \mid i, j \geq 0\}.$$

显然 R 是正则语言, 因为它可由正则表达式

$$10^* = 10^* + 0$$

描述。

若 ADD 是正则语言, 则

$$L = ADD \cap R$$

也应是正则语言。但在 R 中, 字符串具有形式

$$10^i = 10^j + 0.$$

其表示的等式为

$$2^i = 2^j + 0.$$

该等式成立当且仅当 $i = j$ 。因此

$$L = \{10^n = 10^n + 0 \mid n \geq 0\}.$$

下面证明 L 非正则。假设 L 正则，令 p 为其泵长度。取

$$s = 10^p = 10^p + 0 \in L.$$

由泵引理，存在分解 $s = xyz$ ，满足

$$|xy| \leq p, \quad |y| > 0,$$

且对所有 $k \geq 0$ ，都有 $xy^kz \in L$ 。

由于 $|xy| \leq p$ ，子串 y 完全落在第一个 10^p 中。令 $k = 0$ ，即删除 y 。这样得到的 xz 会改变等号左边的二进制数，而等号右边的 $10^p + 0$ 不变。因此 xz 不可能仍具有形式

$$10^n = 10^n + 0.$$

所以

$$xz \notin L,$$

这与泵引理矛盾。故 L 非正则。

于是若 ADD 正则，则正则语言 R 与它的交 L 应正则；但 L 非正则，矛盾。因此

ADD

不是正则语言。■

解释。 这个证明的关键是用正则语言 R 把 ADD 限制到一种很简单的加法：

$$10^i = 10^j + 0.$$

此时加法本身没有复杂性，因为右边只是加 0；唯一要检查的是左右两个二进制数是否完全相同，即是否 $i = j$ 。这就把问题化为

$$\{10^n = 10^n + 0 \mid n \geq 0\},$$

而有限自动机无法记住等号左边有多少个 0，再在等号右边逐一比较。因此该语言非正则，从而 ADD 也不可能正则。

Chapter 2

2.4

2.6

2.8

2.10

2.14

2.16

2.18

2.19

2.22

2.24

2.26

2.32

2.33

2.44

2.45

Chapter 3

3.6

3.8

3.10

3.12

3.15

3.18

3.20

Chapter 4

4.3

4.4

4.6

4.10

4.12

4.15

4.17

4.23

4.27

Chapter 5

5.2

5.5

5.9

5.13

5.17

5.21

5.22

5.24

5.28

5.33

Chapter 6

6.3

6.4

6.7

6.10

6.12

6.14

6.17

6.22

Chapter 7

7.1

7.1 下面各项是真是假?

- | | |
|---------------------------------------|-------------------------------------|
| a. $2n = O(n)$ | ^A d. $n \log n = O(n^2)$ |
| b. $n^2 = O(n)$ | e. $3^n = 2^{O(n)}$ |
| ^A c. $n^2 = O(n \log^2 n)$ | f. $2^{2^n} = O(2^{2^n})$ |

图 33

解答

(a) $2n = O(n)$ ——真. 取常数 $c = 2, n_0 = 1$. 对所有 $n \geq n_0$, 有

$$2n \leq 2 \cdot n = c \cdot n,$$

因此 $2n = O(n)$ 。

(b) $n^2 = O(n)$ ——假. 假设存在常数 $c > 0$ 和 n_0 使得对所有 $n \geq n_0$ 有 $n^2 \leq cn$ 。两边同除以 n (此处 $n > 0$), 得

$$n \leq c,$$

但 n 可以任意大, 这与 c 为固定常数矛盾。故 $n^2 \neq O(n)$ 。

(c) $n^2 = O(n \log^2 n)$ ——假. 考察极限

$$\lim_{n \rightarrow \infty} \frac{n^2}{n \log^2 n} = \lim_{n \rightarrow \infty} \frac{n}{\log^2 n} = +\infty.$$

比值趋于无穷, 说明不存在常数 c 使得 $n^2 \leq c \cdot n \log^2 n$ 对足够大的 n 恒成立。故 $n^2 \neq O(n \log^2 n)$ 。

(d) $n \log n = O(n^2)$ ——真. 对所有 $n \geq 1$, 有 $\log n \leq n$, 因此

$$n \log n \leq n \cdot n = n^2.$$

取 $c = 1, n_0 = 1$ 即可。故 $n \log n = O(n^2)$ 。

(e) $3^n = 2^{O(n)}$ ——真. 利用换底公式

$$3^n = (2^{\log_2 3})^n = 2^{n \log_2 3}.$$

由于 $\log_2 3 \approx 1.585$ 是常数, 故 $n \log_2 3 = O(n)$, 即存在常数 $c = \log_2 3$ 使得 $n \log_2 3 \leq cn$ 。因此 $3^n = 2^{O(n)}$ 。

(f) $2^{2^n} = O(2^{2^n})$ ——真. 任何函数 $f(n)$ 都满足 $f(n) = O(f(n))$: 取 $c = 1, n_0 = 1$, 显然

$$2^{2^n} \leq 1 \cdot 2^{2^n}.$$

故 $2^{2^n} = O(2^{2^n})$ 。

案：结合 Big-O 和 Small-O 记号的定义，可无惑。

7.2

7.2 下面各项是真是假？

a. $n = o(2n)$

b. $2n = o(n^2)$

^A c. $2n = o(3^n)$

^A d. $1 = o(n)$

e. $n = o(\log n)$

f. $1 = o(1/n)$

图 34

小 o 的定义回顾. $f(n) = o(g(n))$ 当且仅当

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0.$$

也就是说, f 相对于 g 是“真正可忽略”的; 仅仅相差常数倍并不满足小 o 关系。

解答

(a) $n = o(2n)$ ——假.

$$\lim_{n \rightarrow \infty} \frac{n}{2n} = \frac{1}{2} \neq 0.$$

n 与 $2n$ 只相差常数因子, 属于同阶, 不满足小 o 关系。

(b) $2n = o(n^2)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{2n}{n^2} = \lim_{n \rightarrow \infty} \frac{2}{n} = 0.$$

故 $2n = o(n^2)$ 。

(c) $2^n = o(3^n)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{2^n}{3^n} = \lim_{n \rightarrow \infty} \left(\frac{2}{3}\right)^n = 0,$$

因为底数 $2/3 < 1$ 。故 $2^n = o(3^n)$ 。

(d) $1 = o(n)$ ——真.

$$\lim_{n \rightarrow \infty} \frac{1}{n} = 0.$$

故 $1 = o(n)$ 。

(e) $n = o(\log n)$ ——假. 由洛必达法则 (或基本增长速度比较)

$$\lim_{n \rightarrow \infty} \frac{n}{\log n} = +\infty.$$

比值发散而非趋于 0, 故 $n \neq o(\log n)$ 。事实上关系恰好相反: $\log n = o(n)$ 。

(f) $1 = o(1/n)$ ——假.

$$\lim_{n \rightarrow \infty} \frac{1}{1/n} = \lim_{n \rightarrow \infty} n = +\infty.$$

比值发散, 故 $1 \neq o(1/n)$ 。正确的关系是 $1/n = o(1)$ 。

案：结合 Big-O 和 Small-O 记号的定义，可无惑。

7.5

7.5 下面的公式是可满足的吗？

$$(x \vee y) \wedge (x \vee \bar{y}) \wedge (\bar{x} \vee y) \wedge (\bar{x} \vee \bar{y})$$

图 35

解答

枚举验证。只有两个变量 x, y ，共有 $2^2 = 4$ 种真值赋值。下表枚举全部可能：

x	y	$x \vee y$	$x \vee \bar{y}$	$\bar{x} \vee y$	$\bar{x} \vee \bar{y}$	φ
0	0	0	1	1	1	0
0	1	1	0	1	1	0
1	0	1	1	0	1	0
1	1	1	1	1	0	0

对于每一组赋值，总能找到至少一个子句的取值为 0，于是整个合取式为 0。

结构化解释。 两变量的二元析取子句一共恰好有 4 个，而每个子句 $(\ell_1 \vee \ell_2)$ 只在令 ℓ_1, ℓ_2 同时为假时取 0，即恰好排除 $\{0, 1\}^2$ 中的一个赋值：

$$\begin{aligned} (x \vee y) : \text{排除}(x, y) &= (0, 0), \\ (x \vee \bar{y}) : \text{排除}(x, y) &= (0, 1), \\ (\bar{x} \vee y) : \text{排除}(x, y) &= (1, 0), \\ (\bar{x} \vee \bar{y}) : \text{排除}(x, y) &= (1, 1). \end{aligned}$$

这四个子句合取起来，恰把 $\{0, 1\}^2$ 的全部四种赋值全部排除，因此没有任何赋值能令公式为真。

结论。

$$\varphi \equiv 0,$$

即 φ 是一个矛盾式，**不可满足** (unsatisfiable)。

7.7

7.7 证明 NP 在并和连接运算下封闭。

图 36

即：若 $A, B \in \mathbf{NP}$ ，则 $A \cup B \in \mathbf{NP}$ 且 $A \circ B \in \mathbf{NP}$ 。

总体思路。 由 $A, B \in \mathbf{NP}$ ，存在非确定型图灵机 N_A 和 N_B ，分别在多项式时间内判定 A 和 B 。设 N_A 的运行时间为 $O(n^{k_1})$ ， N_B 的运行时间为 $O(n^{k_2})$ ，其中 k_1, k_2 为常数。我们分别构造两个非确定型图灵机 $N_{\cup}$ 与 $N_{\circ}$ 。判定 $A \cup B$ 与 $A \circ B$ 。

第一部分: 并封闭

构造 $N_{\cup}$. 对输入 w :

1. 非确定地在以下两个分支中选择其一:

- **分支 1:** 在 w 上模拟 N_A ; 若 N_A 接受则接受, 否则拒绝。
- **分支 2:** 在 w 上模拟 N_B ; 若 N_B 接受则接受, 否则拒绝。

正确性.

$$w \in A \cup B \iff w \in A \text{ 或 } w \in B \iff N_A \text{ 接受 } w \text{ 或 } N_B \text{ 接受 } w.$$

而 $N_{\cup}$ 接受 w 当且仅当其某个非确定分支接受 w , 因此 $N_{\cup}$ 恰好判定 $A \cup B$ 。

时间分析. 每个分支的运行时间为 $O(n^{k_1})$ 或 $O(n^{k_2})$, 因此 $N_{\cup}$ 的运行时间为 $O(n^{\max(k_1, k_2)})$, 仍为多项式。故 $A \cup B \in \mathbf{NP}$ 。

第二部分: 连接封闭

回忆连接的定义

$$A \circ B = \{xy \mid x \in A \text{ 且 } y \in B\}.$$

构造 $N_{\circ}$. 对输入 $w = w_1w_2 \cdots w_n$:

1. **非确定地猜切分点** $i \in \{0, 1, \dots, n\}$, 令 $x = w_1w_2 \cdots w_i, y = w_{i+1}w_{i+2} \cdots w_n$ (当 $i = 0$ 时 $x = \varepsilon$; 当 $i = n$ 时 $y = \varepsilon$)。
2. 在 x 上模拟 N_A 。
3. 在 y 上模拟 N_B 。
4. 若 N_A 接受 x 且 N_B 接受 y , 则接受; 否则在此分支拒绝。

正确性. $w \in A \circ B$ 当且仅当存在切分 $w = xy$ 使 $x \in A$ 且 $y \in B$ 。由非确定性, $N_{\circ}$ 的某条分支恰能猜到正确的 i ; 而在该分支里 N_A 有接受路径判定 $x \in A, N_B$ 有接受路径判定 $y \in B$ 。综合三重非确定性 (切分点 + N_A 的分支 + N_B 的分支), $N_{\circ}$ 接受 w 当且仅当 $w \in A \circ B$ 。

时间分析. 猜切分点的代价为 $O(\log n)$ (记录一个 $\{0, \dots, n\}$ 之间的整数), 可放宽记为 $O(n)$ 。模拟 N_A 于 x 的代价为 $|x|^{k_1} \leq n^{k_1}$, 模拟 N_B 于 y 的代价为 $|y|^{k_2} \leq n^{k_2}$ 。因此 $N_{\circ}$ 总运行时间为

$$O(n + n^{k_1} + n^{k_2}) = O(n^{\max(k_1, k_2)}),$$

仍为多项式。故 $A \circ B \in \mathbf{NP}$ 。 ■

7.9

无向图中的三角形是一个 3 团。证明 $\text{TRIANGLE} \in P$, 其中 $\text{TRIANGLE} = \{\langle G \rangle \mid G \text{ 包含一个三角形}\}$ 。

证明

要证明 $\text{TRIANGLE} \in P$, 只需构造一个确定性多项式时间算法, 能够正确判定任意无向图 G 是否包含三角形。

算法构造

设输入无向图 $G = (V, E)$, 其中 $|V| = n$, $|E| = m$ 。以邻接矩阵表示 G , 则边的存在性查询为 $O(1)$ 。

Algorithm 1 判定 G 是否包含三角形的算法 M

Require: 无向图 $G = (V, E)$ 的邻接矩阵表示

Ensure: 若 G 包含三角形则接受, 否则拒绝

```

1: for all 互不相同的顶点三元组  $(u, v, w) \in V^3$ ,  $u < v < w$  do
2:   if  $(u, v) \in E$  且  $(v, w) \in E$  且  $(u, w) \in E$  then
3:     接受
4:   end if
5: end for
6: 拒绝

```

正确性证明

($\Rightarrow$) **充分性:** 若 G 包含三角形, 设该三角形的三个顶点为 $u, v, w \in V$, 则由三角形定义知:

$$(u, v) \in E, \quad (v, w) \in E, \quad (u, w) \in E.$$

算法在枚举三元组时必然枚举到 (u, v, w) , 此时三个条件全部成立, 算法**接受**。

($\Leftarrow$) **必要性:** 若算法接受, 则存在某三元组 (u, v, w) 使得三条边:

$$(u, v) \in E, \quad (v, w) \in E, \quad (u, w) \in E$$

均成立。由三角形的定义, $\{u, v, w\}$ 构成 G 中的一个三角形。

综合两方面, 算法正确判定 TRIANGLE。

时间复杂度分析

- **枚举三元组:** 从 n 个顶点中选取 3 个互不相同的顶点, 共有

$$\binom{n}{3} = \frac{n(n-1)(n-2)}{6} = O(n^3)$$

个三元组。

- **单次检查:** 对每个三元组, 使用邻接矩阵检查三条边是否存在, 每次查询为 $O(1)$, 三条边共 $O(1)$ 。
- **总复杂度:** 算法总时间复杂度为

$$O(n^3) \cdot O(1) = O(n^3).$$

$O(n^3)$ 是关于输入规模 n 的多项式, 故算法 M 是一个**确定性多项式时间算法**。

案: 证明问题属于 P, 即能够给出一个多项式时间的算法, 并分析时间负责度。

7.10

证明 ALL_{DFA} 属于 P。

证明

记

$$\text{ALL}_{DFA} = \{\langle A \rangle \mid A \text{ 是一个 DFA, 且 } L(A) = \Sigma^*\}.$$

要证明 $\text{ALL}_{DFA} \in P$, 只需构造一个确定性多项式时间判定算法。

关键思路：将问题转化为图的可达性问题。 设

$$A = (Q, \Sigma, \delta, q_0, F)$$

是一个 DFA。注意到

$$L(A) = \Sigma^* \iff A \text{ 接受每一个字符串} \iff A \text{ 没有任何可达的拒绝状态.}$$

等价地,

$$L(A) \neq \Sigma^* \iff \text{存在某个状态 } q \in Q \setminus F, \text{ 使得 } q \text{ 从 } q_0 \text{ 可达.}$$

将状态转移函数 δ 视为一张有向图 G_A : 顶点集为 Q , 对每个 $q \in Q$ 和每个 $a \in \Sigma$, 从 q 向 $\delta(q, a)$ 连一条边。于是, 判定 $\langle A \rangle \in \text{ALL}_{DFA}$ 就等价于判断: 是否存在一个从 q_0 可达且不属于 F 的顶点。

算法构造

Algorithm 2 判定 $\langle A \rangle \in \text{ALL}_{DFA}$ 的算法 M

Require: DFA $A = (Q, \Sigma, \delta, q_0, F)$

Ensure: 若 $L(A) = \Sigma^*$ 则接受, 否则拒绝

- 1: **标记**初始状态 q_0
 - 2: **repeat**
 - 3: 对每个已标记状态 q 和每个字符 $a \in \Sigma$, **标记**状态 $\delta(q, a)$
 - 4: **until** 没有新状态被标记
 - 5: **if** 存在某个已标记状态 $q \notin F$ **then**
 - 6: **拒绝** ▷ 存在可达的拒绝状态
 - 7: **else**
 - 8: **接受** ▷ 所有可达状态均为接受状态
 - 9: **end if**
-

正确性证明

($\Rightarrow$) **充分性:** 步骤 1-2 计算出从 q_0 出发在图 G_A 中可达的全部状态集合, 记为 $R \subseteq Q$ 。若 $L(A) = \Sigma^*$, 则对任意 $w \in \Sigma^*$, 都有

$$\delta^*(q_0, w) \in F.$$

因此从 q_0 可达的每个状态都属于 F , 即 $R \subseteq F$ 。于是步骤 3 中不存在已标记的拒绝状态, 算法接受。

($\Leftarrow$) **必要性:** 若 $L(A) \neq \Sigma^*$, 则存在某个字符串 $w \in \Sigma^*$ 被 A 拒绝。令

$$q = \delta^*(q_0, w),$$

则 $q \notin F$, 且 q 由字符串 w 从 q_0 可达, 所以 $q \in R$ 。因此步骤 3 一定会发现某个已标记的拒绝状态并拒绝。故算法接受当且仅当 $L(A) = \Sigma^*$, 算法正确。

时间复杂度分析

设 $n = |Q|$, $k = |\Sigma|$ 。

- **步骤 1:** 标记初始状态 q_0 , 耗时 $O(1)$ 。
- **步骤 2:** 每轮至多检查 n 个已标记状态与 k 个字符的组合, 共 $O(nk)$ 次转移查询; 而轮数至多为 n , 因为每轮若继续执行, 至少会新增一个已标记状态。因此这一步总耗时为

$$O(n^2k).$$

- **步骤 3:** 扫描全部已标记状态并判断是否属于 F , 耗时 $O(n)$ 。
- **总复杂度:**

$$O(1) + O(n^2k) + O(n) = O(n^2k),$$

这是关于输入规模的多项式时间。

因此, 算法 M 是一个确定性多项式时间算法, 能够正确判定 ALL_{DFA} 。故

$$\text{ALL}_{DFA} \in \mathbf{P}.$$

案: 先把问题翻译成“是否存在可达的拒绝状态”, 再在状态图上做可达性搜索。由于状态数和字母表大小都是有限的, 整个过程在多项式时间内完成。

7.14

7.14 证明 $\mathbf{P}$ 在星号运算下封闭。(提示: 使用动态规划。对于任意的 $A \in \mathbf{P}$, 对输入 $y = y_1 \cdots y_n$, 其中 $y_i \in \Sigma$, 建一个表, 表示对于每个 $i \leq j$, 是否有子串 $y_i \cdots y_j \in A^*$ 。)

图 37

证明

我们证明: 若 $A \in \mathbf{P}$, 则 $A^* \in \mathbf{P}$ 。

设 $A \in \mathbf{P}$, 则存在一个确定性多项式时间判定机 M_A , 对任意长度为 m 的输入, 都能在时间 $O(m^c)$ 内判定其是否属于 A , 其中 c 是某个固定常数。下面构造一个确定性多项式时间算法判定 A^* 。

关键思路: 动态规划。 设输入字符串为

$$y = y_1 y_2 \cdots y_n,$$

其中每个 $y_i \in \Sigma$ 。当 $n = 0$ 时, $y = \varepsilon$, 而 $\varepsilon \in A^*$ 恒成立, 因此可直接接受。

对非空输入, 我们建立一个布尔表 T 。对任意 $1 \leq i \leq j \leq n$, 定义

$$T[i][j] = \begin{cases} \text{true}, & \text{若子串 } y_i y_{i+1} \cdots y_j \in A^*, \\ \text{false}, & \text{否则。} \end{cases}$$

最后只需检查 $T[1][n]$ 是否为真。

递推关系. 子串 $y_i \cdots y_j$ 属于 A^* , 当且仅当它可以写成

$$(y_i \cdots y_{k-1})(y_k \cdots y_j),$$

其中前半段属于 A^* , 后半段属于 A 。这里允许前半段为空串; 当 $k = i$ 时, 前半段就是 ε , 显然属于 A^* 。

因此, 对每个 $i \leq j$ 有

$$T[i][j] = \bigvee_{k=i}^j \left((k = i \text{ 或 } T[i][k-1]) \wedge (y_k \cdots y_j \in A) \right).$$

算法构造

Algorithm 3 判定 A^* 的算法 M^*

Require: 输入字符串 $y = y_1 y_2 \cdots y_n$

Ensure: 若 $y \in A^*$ 则接受, 否则拒绝

```

1: if  $n = 0$  then
2:   接受
3: end if
4: 建立一个  $n \times n$  的布尔表  $T$ , 初值全为 false
5: for  $\ell = 1$  to  $n$  do
6:   for  $i = 1$  to  $n - \ell + 1$  do
7:      $j \leftarrow i + \ell - 1$ 
8:     for  $k = i$  to  $j$  do
9:       if  $k = i$  then
10:         $left \leftarrow \mathbf{true}$ 
11:       else
12:         $left \leftarrow T[i][k-1]$ 
13:       end if
14:       if  $left = \mathbf{true}$  and  $M_A$  接受子串  $y_k \cdots y_j$  then
15:         $T[i][j] \leftarrow \mathbf{true}$ 
16:        break
17:       end if
18:     end for
19:   end for
20: end for
21: if  $T[1][n] = \mathbf{true}$  then
22:   接受
23: else
24:   拒绝
25: end if

```

正确性证明

对长度 $\ell = j - i + 1$ 做归纳, 证明算法算出的每个 $T[i][j]$ 都正确刻画了子串 $y_i \cdots y_j$ 是否属于 A^* 。

基础情形: $\ell = 1$. 此时子串只有一个字符 y_i 。算法会检查唯一的分割点 $k = i$, 此时前半段为空串, 自动视为属于 A^* ; 因此

$$T[i][i] = \mathbf{true} \iff M_A \text{ 接受 } y_i \iff y_i \in A.$$

而长度为 1 的字符串属于 A^* ，只能是它本身就属于 A ，故此时结论成立。

归纳步骤： $\ell \geq 2$ 。假设所有长度小于 ℓ 的子串对应的表项都被正确计算。考虑任意长度为 ℓ 的子串 $y_i \cdots y_j$ 。

根据算法， $T[i][j]$ 被置为 **true** 当且仅当存在某个 $k \in [i, j]$ ，使得

$$(k = i \text{ 或 } T[i][k-1] = \text{true}) \text{ 且 } M_A \text{ 接受 } y_k \cdots y_j.$$

由归纳假设可知，当 $k > i$ 时， $T[i][k-1] = \text{true}$ 等价于 $y_i \cdots y_{k-1} \in A^*$ ；而当 $k = i$ 时，前半段为空串 $\varepsilon \in A^*$ 。同时， M_A 的正确性保证

$$M_A \text{ 接受 } y_k \cdots y_j \iff y_k \cdots y_j \in A.$$

于是，

$$T[i][j] = \text{true} \iff \text{存在某个分割点 } k, y_i \cdots y_{k-1} \in A^* \text{ 且 } y_k \cdots y_j \in A.$$

这正是 $y_i \cdots y_j \in A^*$ 的定义。因此长度为 ℓ 的情形也成立。

由归纳法，所有表项都正确，特别地，

$$T[1][n] = \text{true} \iff y \in A^*.$$

因此算法 M^* 正确判定 A^* 。

时间复杂度分析

- 表 T 一共有 $O(n^2)$ 个表项。
- 对每个表项 $T[i][j]$ ，算法至多枚举 n 个分割点 k 。
- 对每个 k ，至多调用一次 M_A 判定子串 $y_k \cdots y_j$ ；该子串长度不超过 n ，因此耗时为 $O(n^c)$ 。

故总时间复杂度为

$$O(n^2) \cdot O(n) \cdot O(n^c) = O(n^{c+3}),$$

这是关于输入长度 n 的多项式时间。

因此， M^* 是一个确定性多项式时间判定算法，从而

$$A^* \in \mathbf{P}.$$

由于 $A \in \mathbf{P}$ 是任意的，得到

$$A \in \mathbf{P} \implies A^* \in \mathbf{P}.$$

故 $\mathbf{P}$ 在星号运算下封闭。

案：用动态规划记录每个子串是否已经能被分解成若干个属于 A 的片段；这样避免了对所有切分方式做指数级回溯，从而保持了多项式时间。

7.17

7.17 证明若 $\mathbf{P} = \mathbf{NP}$ ，则除了语言 $A = \emptyset$ 和 $A = \Sigma^*$ 以外，所有语言 $A \in \mathbf{P}$ 都是 $\mathbf{NP}$ 完全的。

证明. 采用多项式时间 many-one 归约作为 NP 完全性的定义。假设

$$P = NP.$$

任取语言 $A \in P$, 且

$$A \neq \emptyset, \quad A \neq \Sigma^*.$$

由于 $A \in P \subseteq NP$, 故有

$$A \in NP.$$

下面证明 A 是 NP-hard 的。任取 $L \in NP$ 。由假设 $P = NP$, 可得

$$L \in P.$$

因此存在一个多项式时间算法可以判定 L 。

又因为 $A \neq \emptyset$, 所以存在字符串 $y_1 \in A$; 因为 $A \neq \Sigma^*$, 所以存在字符串 $y_0 \notin A$ 。定义函数 $f: \Sigma^* \rightarrow \Sigma^*$ 为

$$f(x) = \begin{cases} y_1, & x \in L, \\ y_0, & x \notin L. \end{cases}$$

由于 $L \in P$, 可以在多项式时间内判断 $x \in L$ 是否成立; 而 y_0, y_1 是固定字符串, 因此 f 是多项式时间可计算函数。

对任意 $x \in \Sigma^*$, 若 $x \in L$, 则

$$f(x) = y_1 \in A;$$

若 $x \notin L$, 则

$$f(x) = y_0 \notin A.$$

因此

$$x \in L \iff f(x) \in A.$$

所以

$$L \leq_p A.$$

由于 $L \in NP$ 是任意的, 故

$$\forall L \in NP, \quad L \leq_p A.$$

因此 A 是 NP-hard 的。

综上, $A \in NP$ 且 A 是 NP-hard, 所以 A 是 NP 完全的。 $\square$

案: 这道题最值得玩味的地方在于, 规约并不只是“把一个难题机械地变成另一个难题”, 而是在不同问题的表述之间传递同一个判定结构。平时我们把多项式时间规约理解为“把未知问题压到一个已知的标准问题上”; 而在假设 $P = NP$ 之后, 原本“难”与“易”的边界被彻底抹平了。于是, 任何一个既不是空集、也不是全集的语言 $A \in P$, 都可以作为这种传递的落点。证明里只需固定一个属于 A 的字符串 y_1 和一个不属于 A 的字符串 y_0 , 就能把任意 $L \in NP$ 的真假判定压缩成对 A 的真假判定。换句话说, 从一个问题到另一个问题的规约, 可以理解成是在不断切换看问题的视角; 而当这些视角被压缩到极致时, 最后剩下的其实只是最基本的“是或否”的数学判定。

7.23

7.23 令 $CNF_k = \{\langle \phi \rangle \mid \phi \text{ 是一个可满足的 cnf 公式, 其中每个变量最多出现在 } k \text{ 个位置}\}$ 。

- a. 证明 $CNF_2 \in P$ 。
- b. 证明 CNF_3 是 NP 完全的。

图 39

$CNF_k = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 CNF 公式, 且每个变量至多出现 } k \text{ 次}\}.$

(a) 证明 $CNF_2 \in P$. 设输入是一个 CNF 公式 ϕ , 并且其中每个变量在整个公式中出现的总次数至多为 2。记输入长度为 N , 这里 N 可以取为 ϕ 中文字出现的总数。

我们构造如下确定性算法, 用来判定在每个变量至多出现两次的前提下, CNF 公式 ϕ 是否可满足。

Algorithm 4 判定 2-出现 CNF 公式 ϕ 是否可满足的算法

Require: 一个 CNF 公式 ϕ , 其中每个变量至多出现 2 次

Ensure: 若 ϕ 可满足则接受, 否则拒绝

```

1:  $\psi \leftarrow \phi$ 
2: while  $\psi$  中仍含有变量 do
3:   if  $\psi$  中存在空子句 then
4:     拒绝
5:   end if
6:   任取一个仍在  $\psi$  中出现的变量  $x$ 
7:   if  $x$  在  $\psi$  中只正出现, 或只负出现 then
8:     令  $x$  取使这些子句满足的真值
9:     从  $\psi$  中删除所有包含  $x$  或  $\neg x$  的子句
10:  else
11:     $x$  在  $\psi$  中恰好一次正出现、一次负出现
12:    设这两个子句分别为  $C_1 = (x \vee A)$  和  $C_2 = (\neg x \vee B)$ 
13:    从  $\psi$  中删除  $C_1$  和  $C_2$ 
14:    构造归结子句  $D = A \vee B$ 
15:    在  $D$  中删去重复文字
16:    if  $D$  同时含有某个变量  $y$  与  $\neg y$  then
17:      丢弃  $D$  ▷  $D$  为重言式
18:    else if  $D$  为空子句 then
19:      拒绝
20:    else
21:      将  $D$  加入  $\psi$ 
22:    end if
23:  end if
24: end while
25: if  $\psi$  中存在空子句 then
26:   拒绝
27: else
28:   接受
29: end if

```

算法保持的性质. 在算法执行过程中, 始终保持“每个变量至多出现两次”这一性质。

- 在第一种操作中, 我们只是删除一些子句, 因此每个变量的出现次数只会减少。
- 在第二种操作中, 被消去的变量 x 被完全删除。对于其他变量, 它们在新子句 D 中的出现, 仅仅来自原来 A 或 B 中的出现, 因此它们的总出现次数不会增加。

因此每次循环中, 对被选中的变量 x , 确实只会落入算法中的两种情形之一。

正确性证明. 我们证明: 每次循环对公式做的变换都保持可满足性。

第一种情形: x 只以同一种极性出现. 不妨设 x 只正出现。令

$$\psi = C_1 \wedge C_2 \wedge \cdots \wedge C_t \wedge R,$$

其中 $C_1, \dots, C_t$ 是所有含有 x 的子句, R 中不含 x 。

设 ψ' 为从 ψ 中删除 $C_1, \dots, C_t$ 后得到的公式, 也就是算法更新后的公式。

- 若 ψ 可满足, 则取一个满足赋值。把其中 x 的取值改成 1 后, 所有 C_i 仍然满足, 而 R 不受影响。因此该赋值在删去 $C_1, \dots, C_t$ 后仍满足 ψ' 。
- 若 ψ' 可满足, 则把满足 ψ' 的赋值扩张为 $x = 1$, 即可满足所有 C_i , 从而满足 ψ 。

故 ψ 可满足当且仅当 ψ' 可满足。若 x 只负出现, 证明完全类似。

第二种情形: x 一正一负各出现一次. 设

$$\psi = (x \vee A) \wedge (\neg x \vee B) \wedge R,$$

其中 A, B, R 都不含变量 x 。算法将其替换为

$$\psi' = (A \vee B) \wedge R,$$

若 $A \vee B$ 是重言式则直接删除该子句; 若 $A \vee B$ 为空子句则立即拒绝。

下面证明 ψ 与 ψ' 等可满足。

- 若 ψ 可满足, 取其在除 x 之外各变量上的限制赋值。由于

$$(x \vee A) \wedge (\neg x \vee B)$$

被满足, 因此若 $x = 0$ 则必须有 $A = 1$, 若 $x = 1$ 则必须有 $B = 1$ 。无论哪种情况, 都有 $A \vee B = 1$, 故该限制赋值满足 ψ' 。

- 若 ψ' 可满足, 则在满足 ψ' 的赋值下, $A \vee B = 1$ 。若 $A = 1$, 令 $x = 0$, 则 $(\neg x \vee B)$ 自动为真, 且 $(x \vee A)$ 也为真; 若 $A = 0$, 则必有 $B = 1$, 令 $x = 1$ 即可。于是该赋值可扩张为 ψ 的满足赋值。

故 ψ 可满足当且仅当 ψ' 可满足。

由以上两种情形可知, 算法每一步都保持可满足性不变。最终若算法拒绝, 则一定是产生了空子句, 而空子句不可满足; 若算法接受, 则所有变量都已被消去且未出现空子句, 此时剩余公式为空合取, 必可满足。因此算法正确。

时间复杂度分析. 令 N 为输入公式中出现的文字总数。

- 每次循环至少消去一个变量。
- 更重要的是，每次循环都会使当前公式中的文字总数严格下降。
- 在第一种情形中，至少删除一个包含 x 的文字。
- 在第二种情形中，原来的两个子句一共含有

$$|A| + |B| + 2$$

个文字，而加入的新子句最多只含有

$$|A| + |B|$$

个文字，因此文字总数至少减少 2。

所以循环次数至多为 N 次。若采用朴素实现，每次循环中扫描当前公式、定位相关子句并构造归结子句，时间均为 $O(N)$ 。因此总时间复杂度为

$$O(N^2),$$

是多项式时间。

故

$$CNF_2 \in P.$$

(b) 证明 CNF_3 是 NP 完全的。

第一步：证明 $CNF_3 \in NP$. 给定一个真值赋值，我们可以在多项式时间内逐个检查所有子句是否被满足，并统计每个变量在公式中的出现次数是否至多为 3。因此

$$CNF_3 \in NP.$$

第二步：证明 CNF_3 是 NP-hard. 我们从标准的

$$CNF-SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 CNF 公式}\}$$

做多项式时间 many-one 归约。

设 ϕ 是任意一个 CNF 公式。对 ϕ 中每个变量 x ，记它在 ϕ 中总共出现 d_x 次。

- 若 $d_x \leq 3$ ，则保持变量 x 不变。
- 若 $d_x > 3$ ，则把这 d_x 次出现分别替换为新的变量

$$x_1, x_2, \dots, x_{d_x}.$$

若原来第 i 次出现是正文字 x ，则替换为 x_i ；若原来第 i 次出现是否定文字 $\neg x$ ，则替换为 $\neg x_i$ 。

为了强制这些新变量取相同真值，对每个满足 $d_x > 3$ 的原变量 x 再加入如下 d_x 个子句：

$$(\neg x_1 \vee x_2) \wedge (\neg x_2 \vee x_3) \wedge \dots \wedge (\neg x_{d_x-1} \vee x_{d_x}) \wedge (\neg x_{d_x} \vee x_1).$$

把经过上述替换后的原子句，与所有新加入的环形子句合起来，得到新公式 ψ 。这就是归约输出。

归约的大小与出现次数控制. 对每个被拆开的变量 x :

- 每个新变量 x_i 在替换后的原公式部分中恰好出现一次。
- 在环形子句中, x_i 恰好正出现一次、负出现一次。

因此每个 x_i 在 ψ 中总共恰好出现 3 次; 而未被拆开的变量本来就至多出现 3 次。因此 ψ 中每个变量至多出现 3 次, 即

ψ 的确满足 CNF_3 定义中的出现次数限制。

同时, 若 ϕ 的总文字数为 N , 则新引入的变量数和子句数都只是 $O(N)$, 故整个构造可在多项式时间内完成。

正确性: 若 ϕ 可满足, 则 ψ 可满足. 设 α 是 ϕ 的一个满足赋值。

- 对于未拆开的变量 x , 令其在 ψ 中保持同样取值。
- 对于被拆开的变量 x , 令

$$x_1 = x_2 = \cdots = x_{d_x} = \alpha(x).$$

这样, 替换后的每个原子句都仍然满足, 因为我们只是把原来某一次出现的 x 或 $\neg x$ 换成了与 $\alpha(x)$ 同值的新变量。另一方面, 每个环形子句

$$(\neg x_i \vee x_{i+1})$$

在上述赋值下都变成

$$(\neg \alpha(x) \vee \alpha(x)),$$

恒为真。因此 ψ 可满足。

正确性: 若 ψ 可满足, 则 ϕ 可满足. 设 β 是 ψ 的一个满足赋值。我们证明: 对每个被拆开的原变量 x , 一定有

$$\beta(x_1) = \beta(x_2) = \cdots = \beta(x_{d_x}).$$

若不然, 则在循环序列

$$x_1, x_2, \dots, x_{d_x}, x_1$$

中, 必存在某个相邻位置满足

$$\beta(x_i) = 1, \quad \beta(x_{i+1}) = 0,$$

其中下标按模 d_x 计算。于是子句

$$(\neg x_i \vee x_{i+1})$$

在 β 下取值为假, 这与 β 满足 ψ 矛盾。故所有 x_i 必同值。

现在定义原公式 ϕ 的赋值 α :

- 对未拆开的变量 x , 令 $\alpha(x) = \beta(x)$ 。
- 对被拆开的变量 x , 令 $\alpha(x) = \beta(x_1)$ 。

由于同一个原变量 x 的所有副本 x_i 在 β 下同值, 故 ϕ 中每个原子句在 α 下的真假, 与 ψ 中对应替换后子句在 β 下的真假完全一致。因为后者都被满足, 所以前者也都被满足。因此 ϕ 可满足。

归约结论. 我们已经证明

$$\phi \in \text{CNF-SAT} \iff \psi \in \text{CNF}_3.$$

因此

$$\text{CNF-SAT} \leq_p \text{CNF}_3.$$

由于 CNF-SAT 是 NP 完全的, 故 CNF_3 是 NP-hard 的。

综合两步, 得到

$$\text{CNF}_3 \in \text{NP} \quad \text{且} \quad \text{CNF}_3 \text{ 是 NP-hard.}$$

因此

$$\text{CNF}_3 \text{ 是 NP 完全的.}$$

7.27

7.27 图的一个着色是给结点指定颜色, 使得相邻的结点不会被指定为同一种颜色。令

$$3\text{COLOR} = \{ \langle G \rangle \mid G \text{ 的结点被三种颜色着色, 使得任何有边相连的两个结点都有不同的颜色} \}$$

证明 3COLOR 是 NP 完全的。(提示: 利用下面三个子图。)

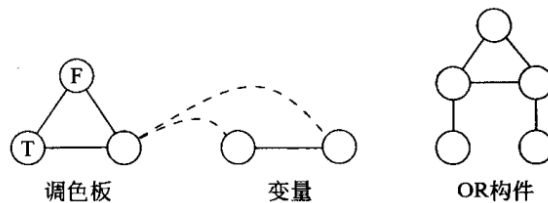


图 40

7.28

7.28 令 $\text{SET-SPLITTING} = \{ \langle S, C \rangle \mid S \text{ 是一个有穷集, } C = \{C_1, \dots, C_k\} \text{ 是由 } S \text{ 的某些子集组成的集合, } k > 0, \text{ 使得 } S \text{ 的元素可以被染为红色或蓝色, 而且对所有 } C_i, C_i \text{ 中的元素不会被染成同一种颜色} \}$ 。证明 SET-SPLITTING 是 NP 完全的。

图 41

7.29

7.29 思考下述计划安排问题。有一份期末考试清单 $F_1, \dots, F_k$ 需要安排时间, 以及学生清单 $S_1, \dots, S_l$ 。每个学生都选择了这些考试的某个子集。必须将这些考试安排到各时段中, 使得同一时段中不会有某个学生同时需要参加两门考试。试问, 如果只用 h 个时段, 是否存在合乎要求的计划。将这个问题形式化为一个语言, 并证明它是 NP 完全的。

图 42

7.30

- 7.30** 这个问题来源于单人游戏“扫雷”，并归结到图。令 G 是一个无向图，它的每个结点要么有一个隐藏的雷，要么是空的。游戏者一个一个地选择结点。如果游戏者选择了有雷的结点，则失败。如果选择了一个空结点，则可以知道该结点的相邻结点中共有多少雷。（相邻结点即同该结点有边相连接的结点。）如果所有的空结点都被选中了，则游戏者获胜。水雷相连问题是一个图，图中的有些结点上标记了数字。标记了数字 m 的结点 v 表示它的相邻结点中有 m 个有雷，我们可以据此判定其他结点中是否有雷。将此问题形式化为一个语言，并证明它是 NP 完全的。

图 43

7.32

- 7.32** 令 $U = \{ \langle M, x, \#^t \rangle \mid \text{非确定型图灵机 } M \text{ 在至少一个分支上在 } t \text{ 步内接受输入 } x \}$ 。证明 U 是 NP 完全的。

图 44

7.34

- 7.34** 如果图 G 中有某个结点子集，其他结点都至少与该子集中的某一结点相邻，则该子集称为支配集。令

$$\text{DOMINATING-SET} = \{ \langle G, k \rangle \mid G \text{ 有一个包含 } k \text{ 个结点的支配集} \}$$

通过从 VERTEX-COVER 问题归约到此问题来证明它是 NP 完全的。

图 45

7.36

- 7.36** 证明：若 $P=NP$ ，则任给布尔公式 ϕ ，若 ϕ 是可满足的，则存在多项式时间算法给出一个 ϕ 的满足赋值。（注意：要求提供的算法计算一个函数而非若干函数，除非是 NP 包含的语言。P=NP 的假定说明 SAT 在 P 中，所以其可满足性的验证是多项式时间可解的。但是这个假设没有说明这项验证如何进行，而且验证不能给出满足赋值。必须证明它们一定能够被找到。提示：重复地使用可满足性验证器一位一位地找出满足赋值。）

图 46

7.39

- 7.39** 在库克-列文定理的证明中，定义窗口为单元的 2×3 矩形。说明如果换用 2×2 的窗口，为什么会使证明失效。

图 47

7.40

7.40 考虑下面的算法 *MINIMIZE*，以 DFA M 为输入，输出 DFA M' ：
MINIMIZE = “对输入 $\langle M \rangle$ ，其中 $M = (Q, \Sigma, \delta, q_0, A)$ 是 DFA：

图 48

- 1) 把 M 的从起始状态出发不可达的状态全部删除。
- 2) 以 M 的状态为结点，构造下面的无向图 G 。
- 3) 在 G 中的每一个接受状态和每一个非接受状态之间连一条边。另外如下添加补充的边。
- 4) 反复执行下面的步骤，直到 G 中无新边加入为止：
 - 5) 对于 M 中每一对不同的状态 q 和 r ，以及每一个 $a \in \Sigma$ ：
 - 6) 如果 $(\delta(q, a), \delta(r, a))$ 是 G 的一条边，就把边 (q, r) 加入 G 中。
- 7) 对于每个状态 q ，令 $[q]$ 代表状态的集合： $[q] = \{r \in Q \mid q \text{ 和 } r \text{ 在 } G \text{ 中无边相连，包括 } r=q\}$ 。
- 8) 形成一个新的 DFA $M' = (Q', \Sigma, \delta', q'_0, A')$ ，其中 $Q' = \{[q] \mid q \in Q\}$ ，
 (若 $[q] = [r]$ ，则 Q' 中只保留其中一个)；对每个 $q \in Q$ 和 $a \in \Sigma$ 有 $\delta'([q], a) = [\delta(q, a)]$ ； $q'_0 = [q_0]$ ， $A' = \{[q] \mid q \in A\}$ 。
- 9) 输出 $\langle M' \rangle$ 。”
 - a. 证明 M 与 M' 等价。
 - b. 证明 M' 极小，即没有更少状态的 DFA 能识别同样的语言。可以无须证明就使用问题 1.52 的结果。
 - c. 证明 *MINIMIZE* 在多项式时间内运行。

图 49

7.42

7.42 *2cnf* 公式是子句的 AND，其中每个子句是至多两个文字的 OR。令 $2SAT = \{\langle \phi \rangle \mid \phi \text{ 是可满足的 } 2cnf \text{ 公式}\}$ 。证明 $2SAT \in P$ 。

图 50

Chapter 8

8.1

- 8.1** 证明对于任意函数 $f: \mathcal{N} \rightarrow \mathcal{R}^+$, 其中 $f(n) \geq n$, 不论用单带 TM 模型还是用两带只读输入 TM 模型, 所定义的空间复杂性类 $\text{SPACE}(f(n))$ 总是相同的。

图 51

8.3

8.6

8.10

8.13

8.20

8.21

8.24

8.28

8.30

8.33

Chapter 9

9.2

9.7

9.8

9.9

9.12

9.16

9.19

9.23

9.25

Chapter 10

10.2

10.4

10.7

10.8

10.10

10.13

10.19